

# REGENERON ISEF

## 2026

**Advancing Post-Quantum Lattice-based  
Cryptography by Developing Efficient  
Shortest Vector Problem Approaches.**

---

**Addison Carey  
Celbridge Community School  
Co. Kildare, Ireland**

---

**Project Number: SFTD004**

---

**Project Report**

# Table of Contents

|                                                                          |           |
|--------------------------------------------------------------------------|-----------|
| <b>1 Introduction</b>                                                    | <b>4</b>  |
| <b>2 Literature Review</b>                                               | <b>6</b>  |
| 2.1 Theoretical Foundations of Lattice-based Cryptography and the SVP    | 6         |
| 2.2 Classical Exact Approaches                                           | 7         |
| 2.3 Basis Reduction Approaches                                           | 7         |
| 2.4 Heuristic and Randomised Approaches                                  | 7         |
| 2.5 Identified Research Gap                                              | 7         |
| 2.6 Research Question                                                    | 8         |
| 2.7 Hypothesis                                                           | 8         |
| 2.8 Aim, Engineering and Design Goals                                    | 9         |
| <b>3 Methodology</b>                                                     | <b>10</b> |
| 3.1 Novel PRG Algorithmic Development Procedure                          | 11        |
| 3.1.1 Iteration 1                                                        | 11        |
| 3.1.2 Iteration 2                                                        | 16        |
| 3.1.3 Iteration 3                                                        | 21        |
| 3.1.4 Iteration 4                                                        | 26        |
| 3.2 Mathematical Interpretation of the Algorithm                         | 36        |
| 3.2.1 Lattices and the SVP                                               | 36        |
| 3.2.2 Basis Conditioning and its Importance                              | 36        |
| 3.2.3 Iteration 1: Brute-force Enumeration                               | 37        |
| 3.2.4 Iteration 2: PRG Solver Approach                                   | 37        |
| 3.2.5 Iteration 3: Local Search Refinement                               | 37        |
| 3.2.6 Iteration 4: LLL-inspired Reduction                                | 38        |
| 3.2.7 Worked Example: Comparing Brute-force and finalised PRG algorithm. | 40        |
| 3.2.8 How PRG Outperforms Brute-Force at Cryptographic Scales            | 44        |
| 3.2.9 How PCA Works for Visualising Lattices of $n > 3$                  | 49        |
| <b>4 Results</b>                                                         | <b>53</b> |
| 4.1 Vector Norm Distribution and Comparison                              | 53        |
| 4.2 Runtime Analysis                                                     | 55        |
| 4.3 Effect of Basis Conditioning                                         | 56        |
| 4.4 PRG Outperformance Rate                                              | 58        |
| 4.5 PRG Mean and Relative Error Analysis                                 | 58        |
| 4.6 Summary of Observed Patterns                                         | 60        |
| <b>5 Analysis and Discussion</b>                                         | <b>61</b> |
| 5.1 Vector Norms                                                         | 61        |
| 5.2 Algorithm Runtime                                                    | 61        |
| 5.3 Basis Conditioning                                                   | 62        |

|                                                                   |                  |
|-------------------------------------------------------------------|------------------|
| <b>5.4 PRG Outperformance Rates .....</b>                         | <b>63</b>        |
| <b>5.5 Errors of Accuracy .....</b>                               | <b>64</b>        |
| <b>5.6 Interpretation and Comparison .....</b>                    | <b>65</b>        |
| <b><i>6 Threats to Validity (Limitations and Errors).....</i></b> | <b><i>66</i></b> |
| <b><i>7 Future Work and Areas of Improvement.....</i></b>         | <b><i>68</i></b> |
| <b><i>8 Conclusions and Applications .....</i></b>                | <b><i>69</i></b> |
| <b><i>9 Acknowledgements.....</i></b>                             | <b><i>71</i></b> |
| <b><i>10 References and Image Credits.....</i></b>                | <b><i>72</i></b> |
| <b><i>11 Appendices .....</i></b>                                 | <b><i>75</i></b> |

## 1 Introduction

Over the past number of decades, cryptography has played a central role in securing communication, financial transactions, and data storage across various digital systems. Classical public-key cryptographical protocols, for example RSA and elliptic curve cryptography (ECC), rely on number-theoretic problems such as integer factorisation and the discrete logarithm problem (Bernstein, Buchmann, & Dahmen, 2009 ). These problems are considered computationally difficult for conventional computers, thereby ensuring the security of cryptographical protocols (Micciancio & Goldwasser, 2002 ). However, the anticipated emergence of quantum computing threatens these systems. Shor's algorithm when executed on a sufficiently powerful quantum computer, has the capability to efficiently solve the integer factorisation and discrete logarithm problems, rendering widely used classical cryptography insecure (Shor P. , 1997 ). This threat has guided the global transition toward Post-Quantum Cryptography (PQC), culminating in recent standardisation of lattice-based encryption and digital-signature schemes (NIST, 2024). As these standards bridge into real-world deployment, analysing the hardness of underlying problems has emerged crucial for evaluating the long-term security of these schemes (Albrecht, Gheorghiu, Postlethwaite, & Schanck, 2020).

One of the most promising directions in PQC is the use of lattice-based cryptographic schemes. Lattices are regular arrangements of points in Euclidean space defined by integer linear combinations of basis vectors (Micciancio & Goldwasser, 2002 ). Unlike number-theoretic assumptions, many hard problems in lattice theory remain resistant to both classical and quantum algorithms (Regev, 2005). Out of these, the Shortest Vector Problem (SVP) stands out as one of the most fundamental. The SVP asks that given a lattice basis, what is the shortest non-zero vector in that lattice? The difficulty of solving SVP derives the security of several cryptographic primitives, including Learning With Errors (LWE) and NTRU, both of which are leading candidates in the NIST PQC standardisation process (Ajtai M. , 1996) (Regev, 2005) (NIST, 2024).

Regardless of its importance, solving the SVP is computationally challenging. It is NP-hard under standard reductions (Micciancio & Goldwasser, 2002 ), and the problem's difficulty escalates exponentially with the dimension of the lattice. In real-world cryptographic applications, lattices of hundreds or even thousands of dimensions are required to obtain quantified security levels against classical and quantum adversaries (Bernstein, Buchmann, & Dahmen, 2009 ), as reflected in the current post-quantum standards (NIST, 2024). At these scales, exact algorithms for SVP are infeasible. Correspondingly, much attention has turned to the development of approximation algorithms and heuristic approaches which can produce useful, but not always optimal, solutions. These methods typically seek to balance accuracy and computational efficiency.

A notably powerful lattice pre-processing technique is basis reduction, where a given lattice basis is transformed into a reduced basis with shorter and more orthogonal vectors. The Lenstra-Lenstra-

Lovász (LLL) algorithm is the most recognised polynomial-time reduction method, widely used in both cryptography and in experimental studies of SVP (Lenstra, Lenstra, & Lovász, 1982). By adopting basis reduction techniques, the condition number of the lattice basis is improved, which can make subsequent heuristic approaches to SVP more effective. Nonetheless, even with reduction, the problem remains computationally demanding at high dimensions.

The motivation for this research lies in creating a novel heuristic strategy for solving the SVP in a range of lattices, with a view to comprehending strengths, limitations, and potential implications for cryptographic applications. While the cryptographic relevance of SVP lies in high dimensions, structured dimensional experimentation provides an accessible framework for developing and testing algorithmic behaviour, and a deeper insight of how the heuristics cooperate with lattice reduction (Micciancio & Voulgaris, 2010).

This project focuses on the design, development, testing and analysis of a **Pruned Randomised Greedy (PRG) novel heuristic algorithm for the SVP** in a range of lattice dimensions. This algorithm incorporates randomised sampling, pruning strategies, and local search refinement, and tested both on original and reduced bases. The objective of this is to understand how closely structured heuristic strategies can approach exact SVP solutions, how their computational cost scales with dimension, and how they interact with lattice reduction methods. This project does not claim to solve SVP at cryptographically scaled dimensions but rather seeks to investigate the potential of a novel heuristic approach in the tested dimensions as a foundation for scaling and adaption.

The importance of this research additionally lies in its position relative to existing research. In modern lattice-based cryptography, heuristic algorithms are studied because they give practical upper bounds on real-world attack capability, in which these bounds are vital for choosing safe parameters and validating long-term security assumptions. Structured testing of the PRG novel approach in a set range of dimensions serves as a tractable setting where accuracy, stability and scaling behaviour of the developed algorithm can be evaluated against exact solutions (Micciancio & Voulgaris, 2010). Through this lens, this project contributes to PQC security analysis which helps to determine how these strategies behave relative to the hardness assumptions that ground lattice-based encryption.

The expansive significance of this work lies in its contribution to PQC research. By developing and experimenting with novel heuristic methods for the SVP, this project objects to highlight the promise and challenges of lattice-based cryptography, an area that is expected to play a defining role in the future of secure communication in a post-quantum world (Campbell, Groves, & Shepherd, 2020 ).

## 2 Literature Review

The SVP is one of the fundamental problems in lattice theory, both from a theoretical viewpoint and as a foundation of modern cryptography. This literature review is divided into four thematic sections including the theoretical foundations, classical algorithmic approaches, heuristic and randomised approaches, and the identified research gap.

### 2.1 Theoretical Foundations of Lattice-based Cryptography and the SVP

- Definitions: A lattice ( $\mathcal{L}$ ) is a discrete set of points in  $\mathbb{R}^n$  formed by all integer linear combinations of a set of basis vectors ( $B$ ) (Micciancio & Goldwasser, 2002 ) (Figure 1).

$$\mathcal{L}(B) = \left\{ \sum_{i=1}^n \alpha_i b_i \mid \alpha_i \in \mathbb{Z} \right\}$$

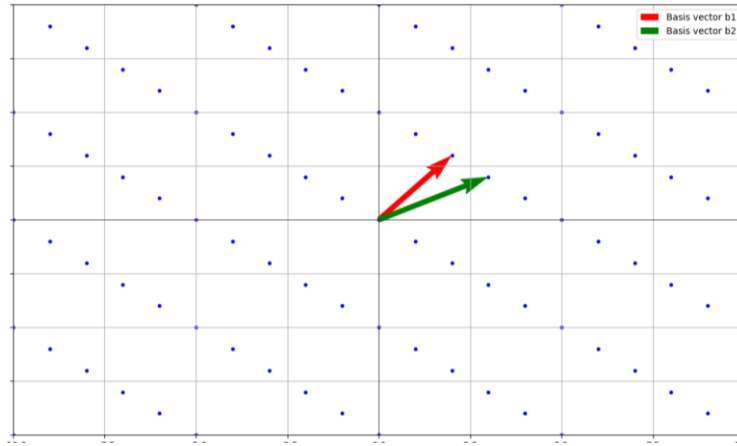


Figure 1: Example of 2D lattice structure with basis vectors, created by Finalist.

The SVP is the problem of finding the shortest non-zero vector in a lattice with respect to the Euclidean norm. It is formally known to be NP-hard under randomised reductions (Ajtai M. , 1996) (Arora, Babai, Stern, & Sweedyk, 1997).

$$\lambda_1(L) = \min_{v \in L \setminus \{0\}} \|v\|$$

- Importance: Lattice-based cryptography constructs cryptographic schemes whose security lies on the computational hardness of lattice problems like the SVP and Closest Vector Problem (CVP). The computational hardness of the SVP additionally grounds encryption, digital signatures, and homomorphic encryption schemes. These problems remain hard even under quantum computation (Micciancio & Regev, 2009) (Regev, 2005) (Peikert, 2016).
- Cryptographic relevance: Lattices of high dimensions (hundreds or thousands) structure the basis for PQC, which objects to resist attacks by quantum algorithms, such as Shor's algorithm (Shor P. W., 1994) (Ajtai & Dwork, 1997).

## 2.2 Classical Exact Approaches

- Enumeration methods: These systematically test lattice vectors with a given radius. This guarantees correctness but expands exponentially, making them unattainable beyond small dimensions (Micciancio & Regev, 2009).
- Voronoi cell algorithms: They partition the lattice into cells to identify the shortest vector (SV) exactly (Nguyen & Stern, 2001). These are exceedingly precise, but computationally expensive.
- The complexity: While exact solvers remain critical for theory and benchmarking, they do not extend to cryptographically relevant dimensions (Gama & Nguyen, 2008).

## 2.3 Basis Reduction Approaches

- LLL reduction: The Lenstra-Lenstra-Lovász reduction (1982) was a breakthrough in making lattice problems approachable (Lenstra, Lenstra, & Lovász, 1982). It presents reduced bases with shorter, more orthogonal vectors. While it does not solve SVP directly, it simplifies computations for any following heuristics or search approaches.
- BKZ (Block Korkine-Zolotarev): This extends LLL with blockwise reduction, yielding bases closer to optimal. This is widely used in cryptographic analysis (Chen & Nguyen, 2011).
- Impact: Basis reduction is now a typical preprocessing step before applying other algorithms.

## 2.4 Heuristic and Randomised Approaches

- Sieving methods: Algorithms like GaussSieve and ListSieve approximate SVP by combining vectors iteratively. They are amongst the strongest heuristics known but demand large memory resources (Micciancio & Regev, 2009).
- Pruning techniques: These diminish enumeration search to promising areas, balancing efficiency with accuracy (Gama & Nguyen, 2008).
- Randomised sampling: Randomised approaches often find short vectors quickly, nevertheless without guarantees (Nguyen & Stern, 2001).
- Observations: These heuristics emphasise that in practice exact optimality is often sacrificed for scalability and speed.

## 2.5 Identified Research Gap

From the reviewed literature, the following primary categories of SVP-solving strategies were highlighted:

1. Exact algorithms like enumeration and Voronoi cell methods become computationally infeasible beyond low dimensions.

2. Basis reduction techniques transform lattice bases into more orthogonal structures which improves the solver performance without directly solving the SVP.
3. Robust heuristics require substantial resources, careful implementation, or parameter tuning.
4. Lightweight and interpretable heuristics that allow experimentation and theoretical exploration are relatively underexplored.

This project aims to contribute to this gap by proposing the novel PRG approach that is resource-efficient, benchmarkable compared against brute-force enumeration solving across a range of lattice dimensions and provides a foundation for further cryptographic research.

## 2.6 Research Question

Regardless of decades of research, the SVP remains a central challenge in computational mathematics and cryptography. Exact algorithms assure correctness but scale exponentially and are infeasible past low dimensions. More advanced techniques have pushed the boundaries of what is computationally obtainable but require steady memory, parameter tuning, or advanced implementations that limit accessibility.

This leaves an open space for novel heuristic methods that incorporate efficiency and practicality, providing approximate solutions that can expand better than brute-force while maintaining competitive accuracy.

Problem statement:

*Can a randomised, pruned and locally refined heuristic algorithm provide an effective and competitive approach to approximating solutions to the SVP, in particular when combined with basis reduction, and how does it function in relation to brute-force enumeration methods across a range of lattice dimensions?*

## 2.7 Hypothesis

If a refined algorithm, local refinement and a basis reduction step is applied, then it is possible to efficiently approximate solutions to the SVP in lattices of differing dimensions which can achieve shorter vectors and lower solver runtimes compared to traditional brute-force enumeration methods. Many classical algorithms for SVP trade efficiency for accuracy (Schnorr & Euchner, 1994) (Gama & Nguyen, 2008). Heuristic methods are often employed as compromises, especially when studying dimensions too small for cryptographic deployment but large enough to provide insights into algorithmic structure. Establishing the PRG heuristic within this spectrum permits for a clearer understanding of its advantages and limitations.



## 2.8 Aim, Engineering and Design Goals

The aim of this project is to propose, enforce and evaluate a novel heuristic algorithm that integrates randomisation, pruning, local refinement and lattice reduction in order to approximate solutions to the SVP. This project further sets to compare its performance against brute-force solvers to assess its scalability as lattice dimensions grow. These aims set out the following design and engineering goals:

1. Algorithm Development: Formulate and implement the novel PRG heuristic for solving the SVP.
2. Preprocessing Integration: Employ basis reduction to improve lattice conditioning and strengthen solver performance.
3. Comparative Evaluation: Test the proposed heuristic algorithm against brute-force search methods across differing lattice dimensions (up to the limits of computational feasibility).
4. Performance Analysis: Judge the effectiveness of the heuristic in terms of solution quality, runtime, and expandability, with particular attention to ill-conditioned lattices.
5. Cryptographic Relevance: Relate findings from small to medium dimensional lattices to their implications for PQC, acknowledging ongoing restraints while accentuating potential directions for higher-dimensional applications.

### 3 Methodology

The methodology for this project was designed to systematically develop, test, and refine an algorithm for approximating solutions to the SVP. The process mixed theoretical analysis with computational experimentation, progressing through a series of iterative models. Each iteration incorporated improvements based on observed performance, culminating in a final novel heuristic that integrates pruning, randomisation, local refinement and lattice preprocessing.

The work followed the agile methodology cycle (Figure 2), typical of computational engineering projects. Agile methodology is a project management approach that enables flexibility and empowers practitioners to engage in rapid iteration. It is based on the Agile Manifesto (Fowler, 2001), a set of values and principles for software development that prioritise the needs of the customer, the ability to respond to change, and the importance of delivering working software regularly. Agile methodologies such as scrum and lean are designed to help teams and individuals deliver high-quality products in a fast-paced and dynamic environment by breaking down complex projects into smaller, more manageable chunks and continually reassessing and adjusting the project plan as needed. Agile teams are typically self-organising and cross-functional and rely on regular communication and collaboration to achieve their goals. This methodology was applied effectively throughout the project:

1. Initial prototypes were created to establish a functional baseline.
2. Successive refinements were introduced, tested, and evaluated.
3. A final version was compared against a benchmark brute-force solver to measure progress toward the engineering and design goals.

All code was written in Python and implemented in a high-level programming environment, while mathematical reasoning and worked examples were used to validate algorithmic choices.

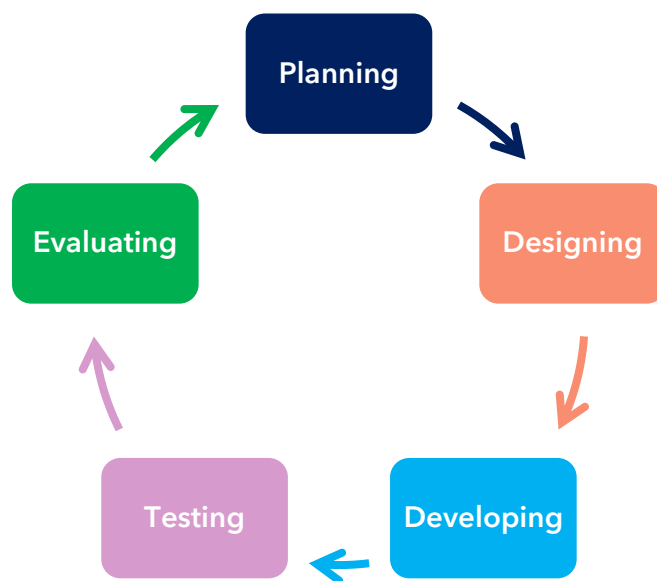


Figure 2: The agile methodology cycle diagram, created by Finalist.

### 3.1 Novel PRG Algorithmic Development Procedure

The development procedure involved four main phases; planning and designing, developing, testing, and evaluation. Each iteration progressively advanced the features and capabilities of the PRG algorithm.

#### 3.1.1 Iteration 1

##### Plan and Design Rationale

The purpose of iteration 1 was to establish a baseline solver for the SVP (Figure 3). This provided a controlled environment against which the subsequent heuristic approach could be evaluated. The brute-force method was selected because it guarantees correctness in low-dimensional lattices, even though it becomes increasingly computationally infeasible as the dimension increases.

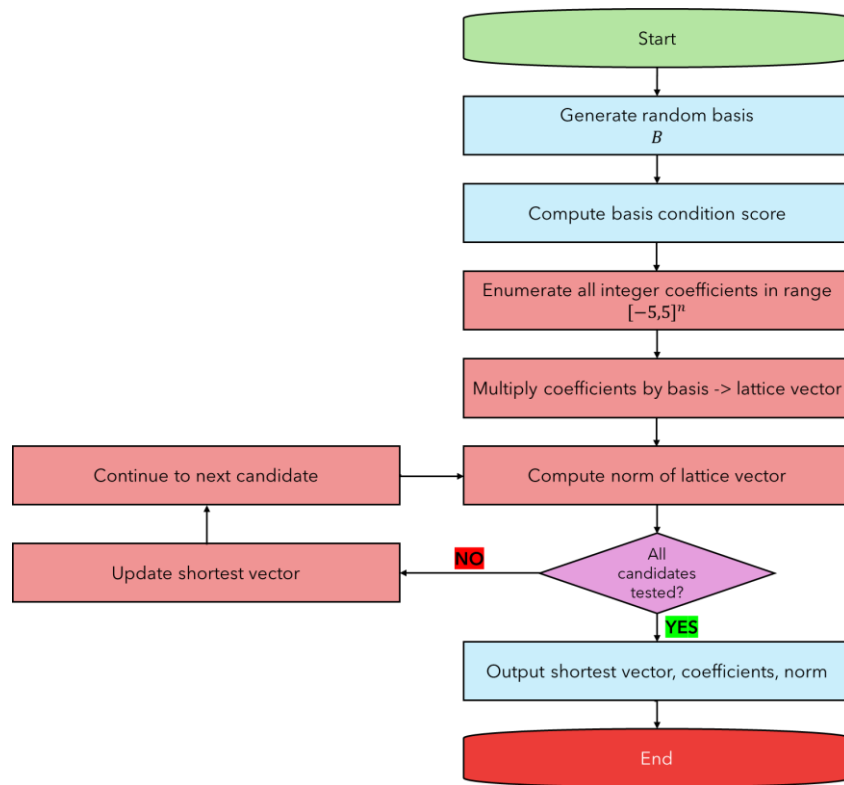


Figure 3: Flowchart for iteration 1 code, created by Finalist.

##### Development

Python was selected as the implementation environment because it is broadly used in computational mathematics, supports efficient vectorised linear-algebra operation, and enables a transparent and reproducible environment. As this project focused on algorithmic behaviour and mathematical scaling, all algorithms were executed in the same computational space to ensure that performance reflected the algorithmic properties, rather than language-specific optimisation. Iteration 1 program was written as a structured Python script. Below is a breakdown of its key components. All screenshots of code below were taken by Finalist.

### 1. Imports and setup:

```
1 #imports AC
2 import numpy as np
3 from sklearn.decomposition import PCA
4 import plotly.graph_objs as go
```

From the imports, *numpy* was used for matrix operations. *Principal Component Analysis (PCA)* from *scikit-learn* reduced high-dimensional data to three dimensions for visualisation purposes. Additionally *plotly* was used to handle 3D plotting to display the lattice.

### 2. Basis condition score function:

```
7 #simple measure of basis conditioning AC
8 def basis_condition_score(B):
9     B_T_B = B.T @ B
10    eigvals = np.linalg.eigvals(B_T_B)
11    return np.max(eigvals) / np.min(eigvals)
```

The basis condition score function quantified the conditioning of a lattice basis by computing the eigenvalues of  $B^T B$  and returning their ratio. A higher score implied a poorly conditioned basis (vectors nearly linearly dependent), while a lower score implied a better-conditioned, suggesting more orthogonal bases. This score became a significant diagnostic throughout the course of the project.

### 3. Brute-force enumeration SVP solver:

```
13 #naive brute-force svp solver AC
14 def naive_svp_solver(B, max_range=5):
15     n = B.shape[1]
16     shortest_vec = None
17     min_norm = float('inf')
```

The solver began by extracting the dimension of the basis  $n$ . It initialised *shortest\_vec* to *None* and set *min\_norm* to infinity, preparing for comparison as candidate vectors were generated.

```
19 #search all integer coefficient vectors AC
20 for coeffs in np.ndindex(*(2*max_range+1 for _ in range(n))):
21     coeffs = np.array(coeffs) - max_range
22     if np.all(coeffs == 0):
23         continue #skip the zero vector AC
```

This loop systematically enumerated all integer coefficient vectors within the hypercube  $[-5,5]^n$ . The zero vector was excluded, since the SVP requires a non-zero SV.

```
24     vec = B @ coeffs
25     norm = np.linalg.norm(vec)
26     if norm < min_norm:
27         min_norm = norm
28         shortest_vec = vec
29         best_coeffs = coeffs
30 return shortest_vec, best_coeffs, min_norm
```

Each coefficient vector was multiplied with the basis  $B$  to generate a lattice vector. The Euclidean norm was computed, and if shorter than the current best, it got stored as the new candidate. At the end, the function returned the SV, coefficients, and norm.

#### 4. Random basis generation:

```
32 #lattice setup AC
33 n = 3 #dimensions
34 B = np.random.randint(-5, 6, size=(n, n)) #random 3D lattice AC
```

A random n-dimensional lattice basis was generated (iteration 1 primarily focused on single runs up to n=5) with integer entries between -5 and 5. Small dimensions were chosen deliberately in this early iteration, to closely monitor how brute-force enumeration grows exponentially with dimension.

#### 5. Solver execution and output:

```
38 #basis quality AC
39 score = basis_condition_score(B)
40 print(f"Basis condition score (higher = worse): {score:.2f}")
41
42 #solve SVP AC
43 shortest_vec, best_coeffs, shortest_norm = naive_svp_solver(B, max_range=5)
```

When the code executed, the conditioning score was calculated first followed by the execution of the brute-force solver.

#### 6. Visualisation:

```
50 #visualization of lattice and SV AC
51 #generate lattice points for plotting AC
52 sample_vectors = []
53 max_range = 5 #consistent with the solver AC
54
55 for coeffs in np.ndindex(*((2*max_range+1,) * n)):
56     coeffs = np.array(coeffs) - max_range #shifts range from [0,10] to [-5,5] AC
57     sample_vectors.append(B @ coeffs)
58
59 sample_vectors = np.array(sample_vectors)
```

A set of lattice points were generated for visualisation by applying the basis to all coefficient combinations within the same range.

```
62 #reduce dimension if needed AC
63 pca_3d = PCA(n_components=3)
64 data_3d = pca_3d.fit_transform(sample_vectors)
65
66 shortest_vec_proj = pca_3d.transform(shortest_vec.reshape(1, -1))
```

PCA projected the lattice points into 3-dimensional space, making them plottable regardless of original dimension. The brute-force SV was also projected for display.

```

68 #plot lattice and SVP AC
69 fig = go.Figure()
70
71 #lattice points on plot AC
72 fig.add_trace(go.Scatter3d(
73     x=data_3d[:, 0], y=data_3d[:, 1], z=data_3d[:, 2],
74     mode='markers',
75     marker=dict(size=3, color='blue'),
76     name='Lattice Points'
77 ))
78
79 #shortest vector on plot AC
80 fig.add_trace(go.Scatter3d(
81     x=[0, shortest_vec_proj[0, 0]],
82     y=[0, shortest_vec_proj[0, 1]],
83     z=[0, shortest_vec_proj[0, 2]],
84     mode='lines+markers',
85     marker=dict(size=5, color='red'),
86     line=dict(width=5, color='red'),
87     name='Shortest Vector'
88 ))
89
90 fig.update_layout(title='SVP Approximation in Random 3D Lattice',
91                   scene=dict(xaxis_title='PCA 1',
92                               yaxis_title='PCA 2',
93                               zaxis_title='PCA 3'),
94                   width=800, height=800)
95
96 fig.show()

```

The lattice points were plotted in blue and the brute-force SV was displayed in red. This step confirmed visually that the algorithm was correctly identifying short lattice vectors.

## Testing

The program was successfully able to identify the shortest vector across multiple randomly generated lattices in a number of dimensions (Figure 4).

- The basis condition score provided a quantitative measure of basis difficulty.
- The SV output returned both the coefficients and the vector norm, validating the correctness of the solver.
- The visual confirmation implementing the PCA-based 3D plots clearly highlighted the SV relative to the initial lattice structure.

```

Basis matrix B:
[[ 4 -2 -3]
 [-2 -3 -5]
 [-1 -3 -2]]
Basis condition score (higher = worse): 32.20

--- Shortest Vector Found ---
Shortest vector: [-1 -2  1]
Coefficients: [ 0 -1  1]
Vector length (norm): 2.449

```

Figure 4: Example output of iteration 1 code and metrics, created by Finalist.

## Evaluation

While brute-force served accurate, the runtime scaled poorly with higher values of  $n$  or  $max\_range$ , confirming the brute-force approach as a valid baseline but unsuitable for scaling effectively. Some of the main limitations identified in this iteration include:

1. Scalability: The exponential runtime growth meant the solver became impractical beyond dimensions 4-5.
2. Cryptographic gap: The lattices in practice require dimensions in the hundreds, highlighting the necessity of heuristics.
3. Dependency on range: The maximum search range parameter ( $max\_range$ ) influenced whether the true SV was found. Increasing it improved accuracy but at the cost of exponential runtime.

In conclusion, these limitations justified developing a **novel heuristic algorithm** in iteration 2.

### 3.1.2 Iteration 2

#### Plan and Design Rationale

Iteration 1 created a baseline brute-force solver for the SVP. While it demonstrated effective in the tested dimensions, brute-force scales exponentially with lattice rank, quickly becoming impractical.

Iteration 2 aimed to address this limitation by introducing a prototype of the novel heuristic PRG solver. Instead of exhaustively searching every possible coefficient vector, PRG randomly samples candidate coefficient combinations within bounded ranges, prunes those that exceed a working radius, and updates this radius dynamically as better solutions are found (Figure 5).

The design goals included:

1. To move from enumeration toward approximate and scalable methods.
2. To implement a solver that could demonstrate reduced runtime compared to brute force in higher dimensions.
3. To integrate both solvers (brute-force and PRG) for direct comparison in terms of speed, solution quality and scalability.

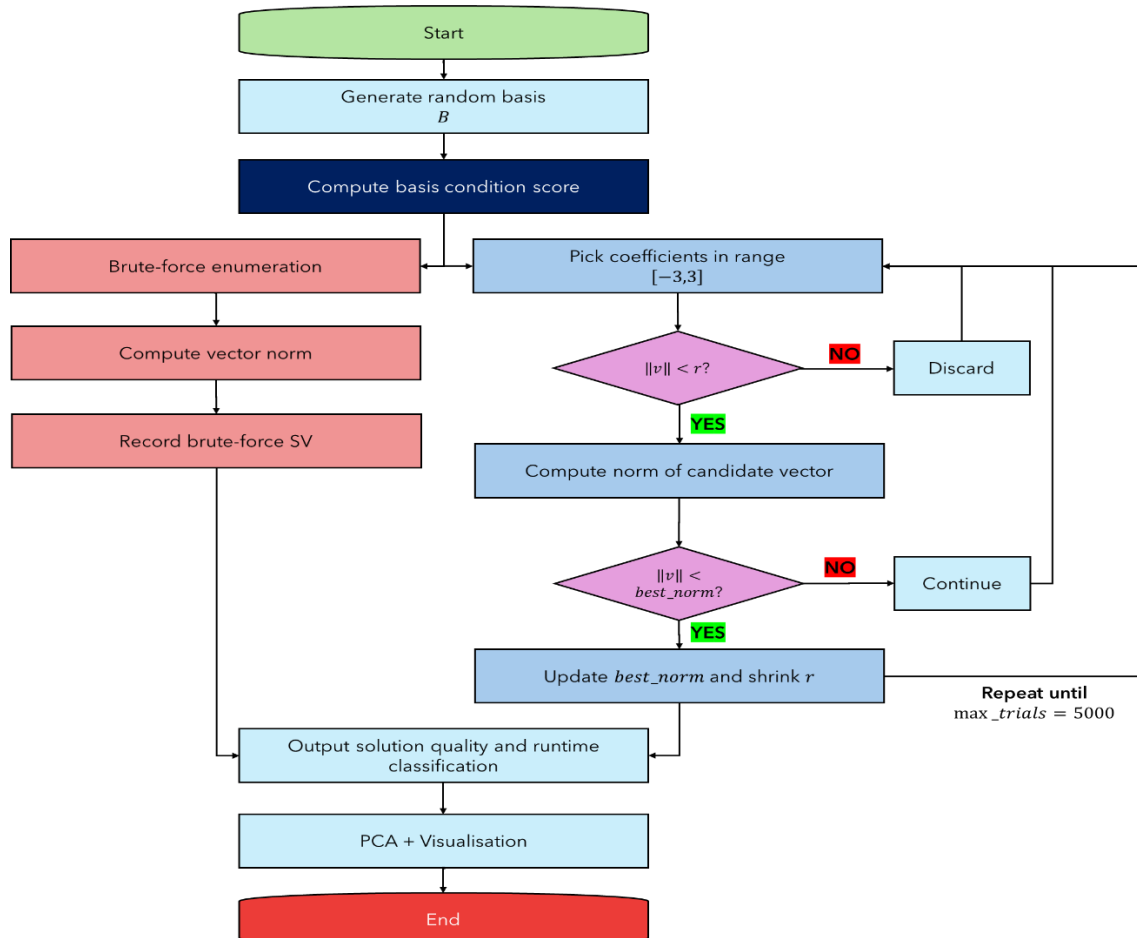


Figure 5: Flowchart of iteration 2 code and PRG framework, created by Finalist.



## Developing

The implementation built upon the iteration 1 framework but introduced several new functions and modifications. All screenshots of code below were taken by Finalist.

### 1. Basis condition score:

```
7 #functions AC
8 def basis_condition_score(B):
9     #simple measure of basis conditioning AC
10    B_T_B = B.T @ B
11    eigvals = np.linalg.eigvals(B_T_B)
12    return np.max(eigvals) / np.min(eigvals)
```

This function remained unchanged from iteration 1 and continued to measure basis conditioning, serving as an indicator as to how poor or well-conditioned the lattice was.

### 2. Brute-force solver:

```
14 def naive_svp_solver(B, max_range=2):
15     #naive brute-force SVP solver AC
16     n = B.shape[1]
17     shortest_vec = None
18     min_norm = float('inf')
19
20     for coeffs in np.ndindex(((2*max_range+1,) * n)):
21         coeffs = np.array(coeffs) - max_range
22         if np.all(coeffs == 0):
23             continue
24         vec = B @ coeffs
25         norm = np.linalg.norm(vec)
26         if norm < min_norm:
27             min_norm = norm
28             shortest_vec = vec
29             best_coeffs = coeffs
30     return shortest_vec, best_coeffs, min_norm
```

This solver additionally remained unchanged, but the search range was reduced to  $\pm 2$  for computational efficiency in 4D lattices. The brute-force solver function iterated through all coefficient vectors in the specified range, excluding the zero vector, and stored the shortest result. This provided a baseline comparison for the PRG heuristic solver.

### 3. PRG solver:

```
32 def prg_svp(B, max_trials=5000, initial_radius=10.0):
33     #PRG SVP solver AC
34     n = B.shape[1]
35     best_vec = None
36     best_norm = float('inf')
37     radius = initial_radius
38
39     for trial in range(max_trials):
40         coeffs = np.random.randint(-3, 4, size=n)
41         if np.all(coeffs == 0):
42             continue
43
44         vec = B @ coeffs
45         norm = np.linalg.norm(vec)
46
47         if norm < radius:
48             if norm < best_norm:
49                 best_norm = norm
50                 best_vec = vec
51                 best_coeffs = coeffs
52                 radius = norm * 1.2 # shrink search focus AC
53
54     return best_vec, best_coeffs, best_norm
```

This element was the primary innovation of iteration 2. The key components include:

- Random sampling: Coefficients are chosen randomly within  $[-3,3]$ .
- Pruning: Candidate vectors are only considered if their length is below the current radius.
- Adaptive radius shrinking: When a shorter vector is found, the search radius is updated ( $radius = norm \times 1.2$ ) to focus future searches near promising regions.
- Iteration control: A maximum number of trials (5000) ensures that runtime remains bounded.

This implementation novelly combined multiple heuristic methods to create an efficient and accurate method for SVP approximation.

#### 4. Lattice setup:

```
56 #lattice setup AC
57 n = 4 #4D lattice AC
58 B = np.random.randint(-5, 5, size=(n, n))
```

The problem space was kept similar to iteration 1 ( $n = 3,4,5$ ), with a particular focus on dimensions where  $n > 3$  to ensure PCA was accurately working.

#### 5. Timing and solver comparison:

```
66 #solve with naive solver AC
67 start_time = time.time()
68 shortest_naive, coeffs_naive, norm_naive = naive_svp_solver(B, max_range=2)
69 time_naive = time.time() - start_time
70
71 #solve with PRG algorithm AC
72 start_time = time.time()
73 shortest_unique, coeffs_unique, norm_unique = prg_svp(B, max_trials=5000, initial_radius=10.0)
74 time_unique = time.time() - start_time
```

Both solvers were run with `time.time()` to record execution times, allowing quantitative evaluation of performance differences.

#### 6. Visualisation with PCA:

The 4D lattice was projected into 3D space using PCA for interpretability. Both brute-force and PRG solutions were plotted alongside the lattice points, using red for brute-force and green for PRG, aiding to visualise differences in identified vectors.

## Testing

Two solvers were tested under identical lattice conditions:

#### 1. Brute-force solver:

- Provided guaranteed correctness within specific range.
- Performance was noticeably decreasing with dimensional increase, especially as dimensions increase.
- Norm of the SV served as a baseline ground truth for comparison

## 2. PRG Solver:

- Typically identified a vector of similar norm to brute-force, with most cases finding a matching vector indicating strong and promising performance.
- Performance varied slightly due to randomness but remained consistent in practice when trial counts were sufficiently large.
- Achieved near-equivalent SV results while completing.

Evaluation metrics (Figure 7):

- Solution quality (length of shortest vector).
- Runtime efficiency (seconds per solver).
- Coefficient outputs (to validate correctness and interpret structure).

The PCA-based 3D plot supported interpretation by (Figure 6):

- Displaying lattice points (blue)
- Showing brute-force SV (red)
- Showing PRG SV (green)

This provided immediate visual evidence that both methods were able to identify comparably short vectors, though their algorithms and processes significantly differ.

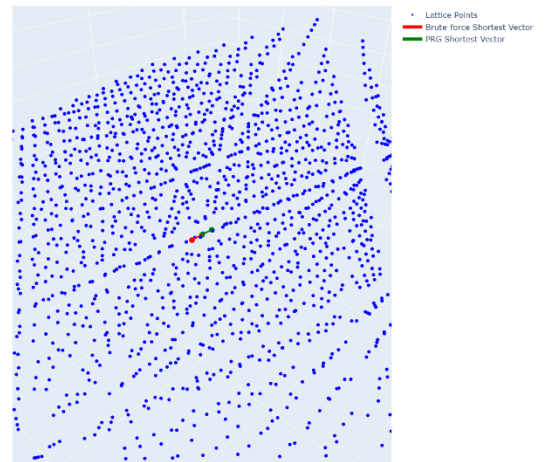


Figure 6: PCA lattice projection in 3D space, created by Finalist.

```
Basis matrix B:
[[-3 -3 1 -1]
 [-3 5 1 -4]
 [ 2 1 3 2]
 [ 5 1 -2 -4]]
Basis condition score (higher = worse): 9.42

--- Brute-Force Solver ---
Shortest vector: [-1 -1 -3 2]
Coefficients: [ 0 0 -1 0]
Norm length: 3.873
Time taken: 0.024 seconds

--- PRG Solver ---
Shortest vector: [ 1 1 3 -2]
Coefficients: [0 0 1 0]
Norm length: 3.873
Time taken: 0.162 seconds
```

Figure 7: Output of iteration 2 results and metrics, created by Finalist.

## Evaluation

Iteration 2 marked a shift from guaranteed correctness to approximate efficiency through the implementation of the novel PRG approach. Below are some of the key evaluations:

- Strengths: The PRG solver was more scalable than brute-force in the exhibited 3-5D lattices.
- Limitations: Random sampling introduces non-determinism and potential failure to find the true shortest vector without sufficient trials. The chosen parameter set ( $\pm 3$  coefficient range, 5000 trials, radius shrink factor 1.2) is somewhat arbitrary and would benefit from additional systematic tuning.
- Transition: This iteration laid the foundation for iterations 3 and 4, where further refinements were introduced to further enhance algorithmic robustness and cryptographic relevance.

### 3.1.3 Iteration 3

#### Plan and Design Rationale

The third iteration built directly upon the iteration 2 prototype of PRG solver by addressing a key limitation. While random sampling with pruning improved efficiency compared to brute-force, it sometimes converged prematurely and struggled to effectively refine solutions.

To mitigate this, iteration 3 integrated a local search refinement stage (Figure 9). After a promising candidate vector was identified through randomised sampling, the algorithm explored its immediate neighbourhood (small coefficient perturbations) to determine if a better SV lied nearby. This hybrid approach balanced exploration (random sampling across the lattice) with exploitation (local refinement around strong candidates).

The main design objectives for iteration 3 were (Figure 8):

1. To maintain efficiency improvements from iteration 2.
2. To add local refinement to enhance solution accuracy.
3. To compare performance against both brute-force and assess improvement from the iteration 2 PRG prototype.

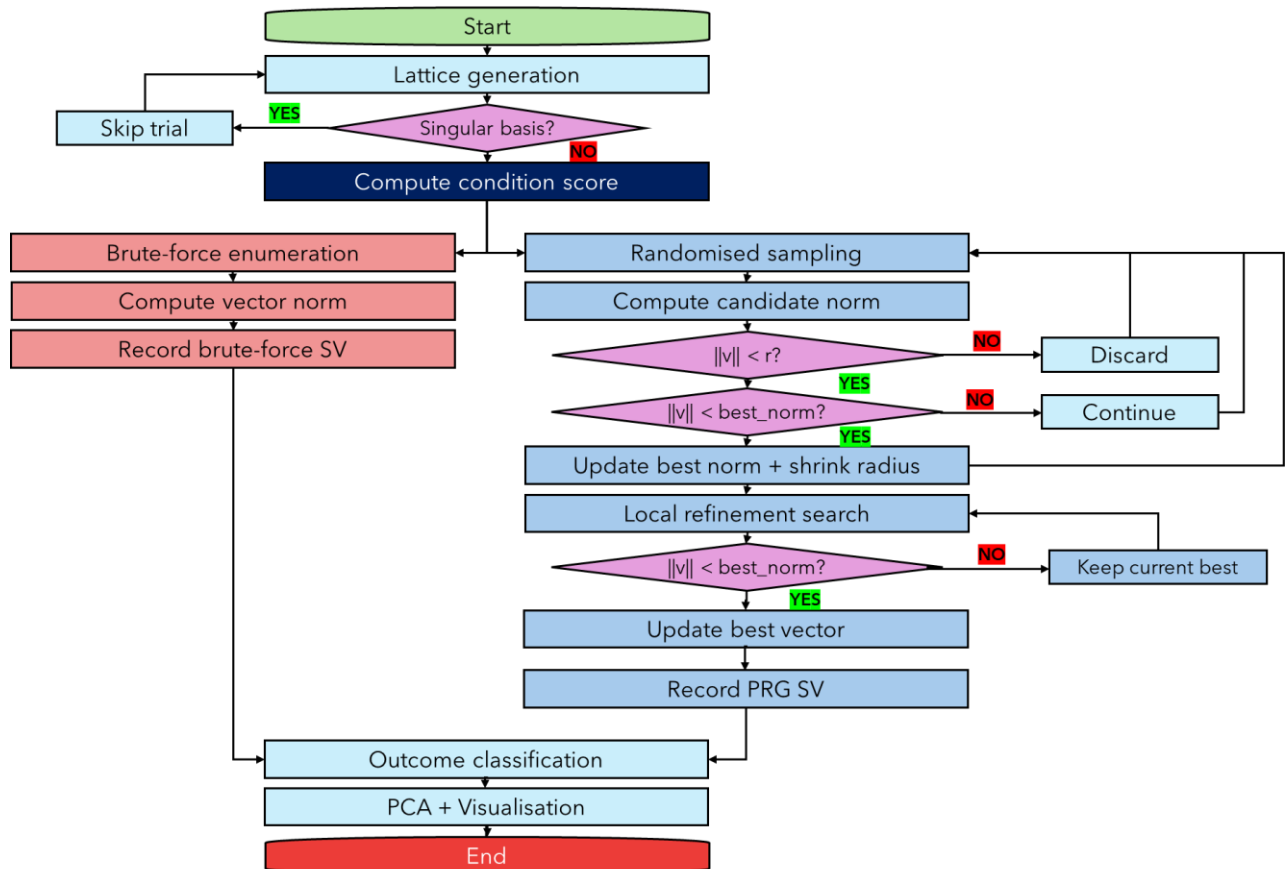


Figure 8: Flowchart of iteration 3 code, created by Finalist.

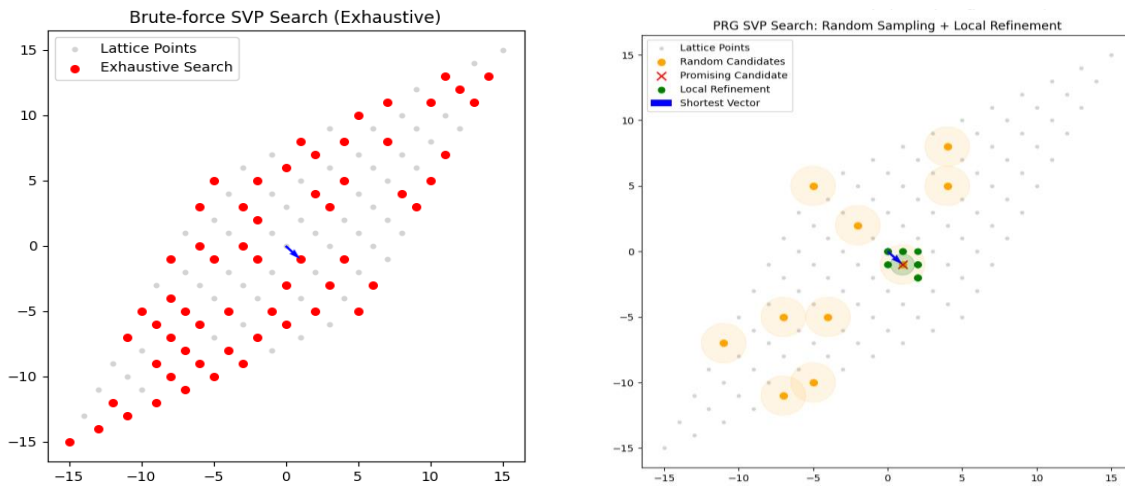


Figure 9: Side-by-side demonstration of brute-force search method VS PRG approach, created by Finalist.

## Development

The implementation followed the same computational framework as previous iterations. All screenshots of code below were taken by Finalist. The main difference stayed in the solver:

- A two-stage process was used:
  - Stage 1 (PRG): Random sampling with pruning identified a strong candidate
  - Stage 2 (Local search refinement): Vector neighbours in coefficient space were tested to refine the solution.
- 1. Imports, basis condition score, brute-force solver:

```

1  #imports AC
2  import numpy as np
3  import time
4  from sklearn.decomposition import PCA
5  import plotly.graph_objs as go
6  # functions AC
7
8  def basis_condition_score(B):
9      #simple measure of basis conditioning AC
10     B_T_B = B.T @ B
11     eigvals = np.linalg.eigvals(B_T_B)
12     return np.max(eigvals) / np.min(eigvals)
13
14  def naive_svp_solver(B, max_range=2):
15     #naive brute-force SVP solver AC
16     n = B.shape[1]
17     shortest_vec = None
18     min_norm = float('inf')
19
20     for coeffs in np.ndindex((2*max_range+1) * n):
21         coeffs = np.array(coeffs) - max_range
22         if np.all(coeffs == 0):
23             continue
24         vec = B @ coeffs
25         norm = np.linalg.norm(vec)
26         if norm < min_norm:
27             min_norm = norm
28             shortest_vec = vec
29             best_coeffs = coeffs
30     return shortest_vec, best_coeffs, min_norm

```

The programming libraries remained consistent with earlier iterations. The basis condition score function also remained unchanged from previous iterations, as it still provided the indication for how well or poorly-conditioned the lattices were. The brute-force solver also remained the same as previous iterations, being used as a baseline for comparison against PRG.

## 2. PRG solver with local refinement:

```
32 def PRG_svp_localsearch(B, max_trials=5000, initial_radius=10.0):
33     #PRG SVP solver + local search refinement AC
34     n = B.shape[1]
35     best_vec = None
36     best_norm = float('inf')
37     radius = initial_radius
```

The local refinement element was the core advancement in iteration 3.

- Stage 1: Random sampling with pruning:

```
39     #random sampling with pruning AC
40     for trial in range(max_trials):
41         coeffs = np.random.randint(-3, 4, size=n)
42         if np.all(coeffs == 0):
43             continue
44
45         vec = B @ coeffs
46         norm = np.linalg.norm(vec)
47
48         if norm < radius:
49             if norm < best_norm:
50                 best_norm = norm
51                 best_vec = vec
52                 best_coeffs = coeffs
53                 radius = norm * 1.2 # shrink search focus AC
```

As in iteration 2, random coefficient vectors were generated and tested. The shrinking radius ensured the search narrowed over time, focusing only on promising areas in the lattice.

- Stage 2: Local search refinement:

```
55     #local search around best_coeffs AC
56     for delta in np.ndindex(*(3,) * n): # explore neighbors in [-1, 0, 1] AC
57         offset = np.array(delta) - 1
58         neighbor = best_coeffs + offset
59         if np.all(neighbor == 0):
60             continue
61         vec = B @ neighbor
62         norm = np.linalg.norm(vec)
63         if norm < best_norm:
64             best_norm = norm
65             best_vec = vec
66             best_coeffs = neighbor
67
68     return best_vec, best_coeffs, best_norm
```

After stage 1 finds the strongest SV candidate, stage 2 systematically tested neighbouring coefficient vectors ( $\pm 1$  in each dimension). This was effectively a gradient-free refinement, improving accuracy without major additional computational overhead.

## 3. Lattice setup and solvers:

```
70     #lattice setup AC
71     n = 4 # 4D lattice AC
72     B = np.random.randint(-5, 6, size=(n, n))
```

The lattice was generated, with primary focus on similar sizes of  $n = 3,4,5$ . These same dimensions were kept as previously, which allowed stronger benchmarking.

```

80 #solve with brute-force solver AC
81 start_time = time.time()
82 shortest_naivebf, coeffs_naivebf, norm_naivebf = naive_svp_solver(B, max_range=2)
83 time_naivebf = time.time() - start_time
84
85 #solve with PRG solver AC
86 start_time = time.time()
87 shortest_prg, coeffs_prg, norm_prg = PRG_svp_localsearch(B, max_trials=5000, initial_radius=10.0)
88 time_prg = time.time() - start_time

```

Both solvers were executed simultaneously, allowing results to be compared side-by-side.

```

90 #print results AC
91 print("\n--- Brute-Force Solver ---")
92 print(f"Shortest vector: {shortest_naivebf}")
93 print(f"Coefficients: {coeffs_naivebf}")
94 print(f"Norm length: {norm_naivebf:.3f}")
95 print(f"Time taken: {time_naivebf:.3f} seconds")
96
97 print("\n--- PRG + Local Search Solver ---")
98 print(f"Shortest vector: {shortest_prg}")
99 print(f"Coefficients: {coeffs_prg}")
100 print(f"Norm length: {norm_prg:.3f}")
101 print(f"Time taken: {time_prg:.3f} seconds")

```

The results output similar to previously included the SV, coefficients, norm length, and runtime. The comparison between both highlighted efficiency and solution accuracy.

#### 4. Visualisation:

As in the previous iteration, PCA was used to transform the lattice structures into 3D for plotting. The brute-force vector was shown in red, and the PRG vector was shown in green. This visual comparison reinforced algorithm effectiveness and solution qualities.

### Testing

Testing for this iteration involved:

1. Executing the brute-force method as a baseline correctness reference.
2. Executing the improved PRG solver applied under the same lattice conditions.
3. Compiling the comparison metrics (Figure 11):
  - SV norm
  - Runtime
  - Robustness and algorithm consistency across different random lattice bases.
4. Testing dimensionality across lattices, with preparation to extend beyond to examine scalability.
5. Validating visualisation with plotly to confirm that the identified vectors were geometrically plausible relative to the lattice points (Figure 10).

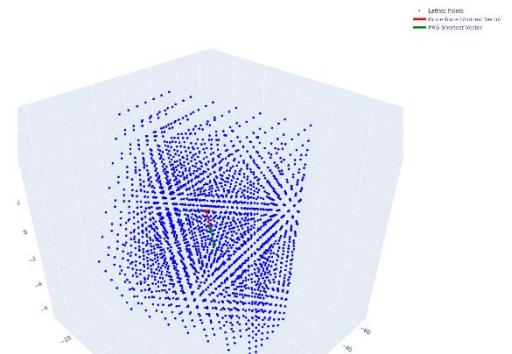


Figure 10: PCA projected lattice into 3D space, created by Finalist.



```
Basis matrix B:
[[-2 -5 -1  2]
 [ 4 -3  4 -3]
 [-1  1 -1  2]
 [ 3 -3  5 -5]]
Basis condition score (higher = worse): 290.00

--- Brute-Force Solver ---
Shortest vector: [-1 -1 -1  0]
Coefficients: [ 0  0 -1 -1]
Norm length: 1.732
Time taken: 0.027 seconds

--- PRG + Local Search Solver ---
Shortest vector: [1 1 1 0]
Coefficients: [0 0 1 1]
Norm length: 1.732
Time taken: 0.236 seconds
```

Figure 11: Output of results and key metrics from iteration 3 code, created by Finalist.

## Evaluation

- Solution accuracy: the improved PRG with local refinement often found vectors matching or improving upon the brute-force solution.
- Efficiency gains: Although higher than brute-force due to overall computational overhead, runtime remained relatively low.
- Refinement benefit: The local search stage corrected cases where the initial PRG prototype alone stalled in suboptimal solutions.
- Scalability: By operating with competitive performance, refinements in this phase showed a greater promise for higher-dimensional testing compared to previous iterations.
- Limitations: The condition of the lattices played a significant role in the solver, as poorly conditioned lattices are less orthogonal, hence more difficult to solve. The effectiveness of the PRG approach significantly depended on this. Additionally, performance in much higher dimensions remained untested.

### 3.1.4 Iteration 4

#### Plan and Design Rationale

By the previous iteration, the PRG algorithm already displayed efficiency gains over brute-force enumeration. However, both solvers still depended heavily on the quality of the input basis. A poorly conditioned basis led to highly skewed lattice vectors, which makes finding a true SV computationally more difficult. This iteration outlined two sets of refinements to gather results effectively and conclusively.

#### Refinement 1:

To address this, iteration 4 built on PRG by introducing a lattice preprocessing step algorithm. Basis conditioning algorithms are a cornerstone of lattice cryptography, in which they transform a lattice basis into a more orthogonal form without changing the lattice itself. The purpose of adding basis conditioning here is to both (Figure 12):

1. Improve solver efficiency: By reducing the overlap between basis vectors, the solver should have an easier time converging to shorter vectors.
2. Demonstrate preprocessing as standard practice: In real-world lattice-based cryptography, preprocessing with basis reduction is standard, so replicating this step makes the project more realistic and applied.

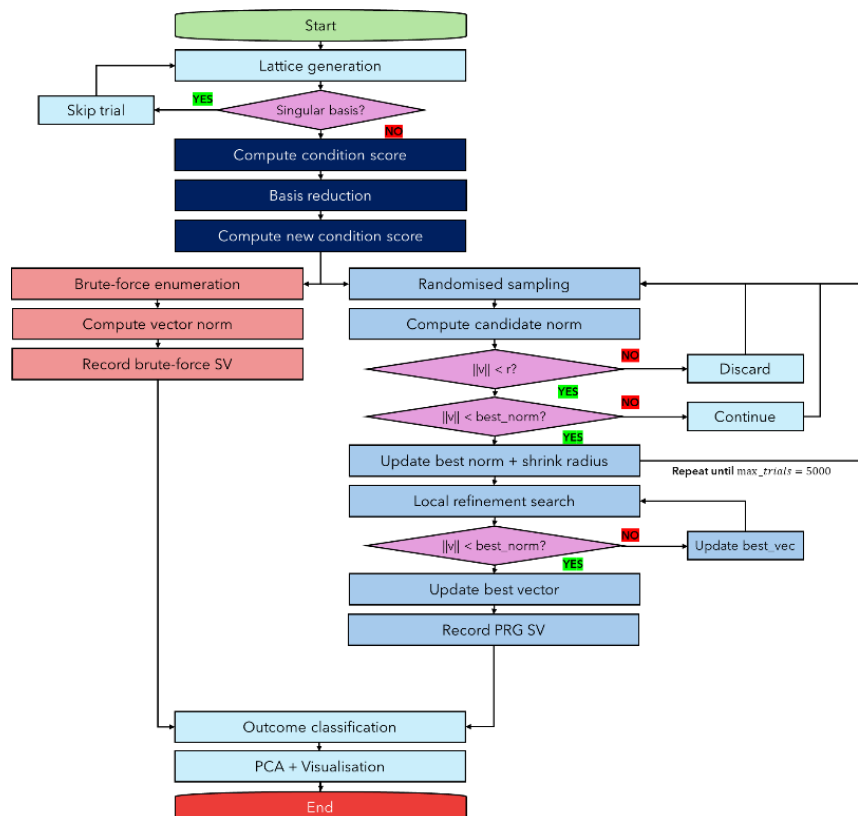


Figure 12: Flowchart of final iteration 4 PRG and brute-force solver code and framework, created by Finalist.

This version of the algorithmic framework was set to be initially tested on a range of dimensions ( $n = 3, 4, 5, 6$ ) across 200 total trials, where both algorithms are given the same lattice and asked to solve for the SVP, allowing results to be compared and analysed.

## Refinement 2:

After the first initial set of testing completed, PRG was set to undergo any minor algorithmic changes to reach its full potential. The main areas of focus for these changes included:

- Final adjustments to the PRG solver framework.
- Refinements to basis conditioning, based on initial performance in the refinement 1 phase,
- Plotly visualisation and interface enhancement.

Additionally, a final testing framework was designed similar to refinement 1, to fully capture the capability of the novel PRG approach in terms of solution quality, runtime, scalability and error. The final results are based on testing PRG against brute-force across dimensions  $n = 2 - 10$  with 1000 trials per dimension. In each trial, PRG and brute-force are given a lattice and solve for the SV. A number of insightful metrics were collected based on this to fully analyse PRG's capabilities.

Finally, a structured mathematical implementation was developed to effectively and accurately show how PRG works in comparison to brute-force on a theoretical level. A mathematical model was also created using log bounds to show how both algorithms grew in cryptographically scaled dimensions, marking the crossover point dimension where PRG becomes more efficient than brute-force. This served incredibly effective as it allowed the accurate interpretation of PRG beyond the current testing limits.

## Development

Iteration 4 introduced several new components compared to previous iterations. Each new addition is described in detail below, with explanations to highlight both functionality and rationale. Basis reduction was introduced during the first refinement, and other minor changes to the framework were introduced after. All screenshots of code below were taken by Finalist.

### 1. Basis reduction:

```
8 #LLL-like reduction AC
9 def lll_reduce(B):
10     B = B.copy().astype(float)
11     n = B.shape[1]
12     for i in range(1, n):
13         for j in range(i):
14             mu = np.dot(B[:, i], B[:, j]) / np.dot(B[:, j], B[:, j])
15             B[:, i] -= round(mu) * B[:, j]
16     return B.astype(int)
```

This function transformed the input basis  $B$  into a ‘reduced’ version. It first made a float copy of  $B$ , since fractional values arise during dot product division. A nested loop iterated over pairs of basis vectors. For each pair  $(i, j)$ , the coefficient was computed. The term  $\text{round}(\mu)$  ensured integer alignment, subtracting multiples of earlier vectors from later ones, reducing overlap and redundancy.

Even though this was a simple version, it demonstrated the key principle of orthogonalization. In lattice cryptography, preprocessing the basis is essential before attempting hard problems, because a badly conditioned basis makes all algorithms less effective.

## 2. Basis condition score tracking:

```

83 #lattice Setup AC
84 n = 4 #dimensions
85 B = np.random.randint(-5, 6, size=(n, n))
86 print("Original Basis Matrix B:\n", B)
87 original_score = basis_condition_score(B)
88 print(f"Condition Score (original): {original_score:.2f}")
89
90 #apply LLL-like reduction prep AC
91 B_reduced = lll_reduce(B)
92 print("\nReduced Basis Matrix B:\n", B_reduced)
93 reduced_score = basis_condition_score(B_reduced)
94 print(f"Condition Score (reduced): {reduced_score:.2f}")

```

This function was used before and after reduction. It quantified how skewed the bases were. The Gram matrix of  $B^T B$  was computed. The eigenvalues of this matrix represented the spread of vector lengths and orientations. The ratio of smallest to largest eigenvalues was returned as the condition score. A lower score indicated the basis vectors are better conditioned, which directly affected solver performance. Applying this function before and after reduction directly measured the improvement.

## 3. Results Table:

```

106 #results Table AC
107 table = [
108     ["Naive Brute-Force", str(coeffs_naivebf), f"{norm_naivebf:.3f}", f"{time_naivebf:.4f} s"],
109     ["PRG + Local Search", str(coeffs_prg), f"{norm_prg:.3f}", f"{time_prg:.4f} s"]
110 ]
111
112 print("\nComparison of Solvers (Iteration 4, LLL-Reduced Basis):")
113 print(tabulate(table, headers=["Method", "Coefficients", "Norm Length", "Runtime"]))

```

Iteration 4 introduced structured tabulation of results (using the *tabulate* library). This allowed a side-by-side comparison of methods across runtime, coefficient selection, and norm length. All visualisation methods were kept the same as previous iterations, displaying the lattice and shortest vector on a PCA plot.

The following key elements were added after the refinement 1 testing was completed. These additions were added as final fixtures to the algorithm.

#### 4. Improved basis reduction:

```

13 # gram schmidt orthogonalisation AC
14 def gram_schmidt(B):
15     G = np.zeros_like(B)
16     mu = np.zeros((n, n))
17     for i in range(n):
18         G[:, i] = B[:, i]
19         for j in range(i):
20             mu[i, j] = np.dot(B[:, i], G[:, j]) / np.dot(G[:, j], G[:, j])
21             G[:, i] -= mu[i, j] * G[:, j]
22     return G, mu
23
24 k = 1
25 while k < n:
26     G, mu = gram_schmidt(B)
27     # size reduction AC
28     for j in range(k - 1, -1, -1):
29         if abs(mu[k, j]) > 0.5:
30             B[:, k] -= round(mu[k, j]) * B[:, j]
31             G, mu = gram_schmidt(B)
32     # L condition AC
33     if np.dot(G[:, k], G[:, k]) >= (delta - mu[k, k-1]**2) * np.dot(G[:, k-1], G[:, k-1]):
34         k += 1
35     else:
36         B[:, [k, k-1]] = B[:, [k-1, k]] # swap the columns AC
37         k = max(k - 1, 1)
38
39 return B.astype(int)

```

The original basis reduction step from refinement 1 was slightly amended from a simple-size reduction approach to a more complex algorithm using Gram-Schmidt orthogonalization, size reduction, and the Lovász condition, making it overall more similar to the LLL basis reduction technique. This ensured the basis vectors were not only shorter but closer to orthogonal, significantly improving the structure of the lattice. More well-conditioned bases reduced distortion in the search space, which allowed the solvers to operate more effectively (Lenstra, Lenstra, & Lovász, 1982) (Nguyen & Stehlé, 2010).

#### 5. Enhanced PRG random sampling:

```

73 for trial in range(max_trials):
74     # random sampling AC
75     coeffs = np.random.normal(0, 1.5, size=n)
76     coeffs = np.round(coeffs).astype(int)
77     if np.all(coeffs == 0):
78         continue

```

The PRG algorithm was modified slightly in terms of its randomisation. The refinement biased the search towards smaller coefficients, which are more likely to produce shorter lattice vectors. As shorter vectors are usually located near the origin, this sampling strategy aligned more closely with the geometry of lattices. This overall improved the efficiency of the search by focusing on more promising regions rather than exploring all directions equally.

#### 6. Refined search radius:

The update rule for the search radius was tightened (1.2 to 1.1) so that it decreased more aggressively when better vectors were found. This meant that once a good SV candidate was identified, the algorithm quickly restricted its search to a smaller region around that solution. As a result, unnecessary exploration

of less promising vectors was reduced, which overall additionally improves efficiency by effectively pruning the search space while maintaining the ability to refine good solutions.

#### 7. Expanded local search:

```

93 # extended local search to check +-1 and +-2 neighbours AC
94 for delta in np.ndindex(*(5,) * n)):
95     offset = np.array(delta) - 2 # -2 to +2 AC
96     neighbour = best_coeffs + offset
97     if np.all(neighbour == 0):
98         continue

```

The local search refinement was slightly extended to examine a wider neighbourhood around the best candidate solution. Instead of only checking immediate neighbours, the algorithm explored a large range of nearby coefficient combinations. This increased the likelihood of identifying improved vectors that may have been missed in the initial random sampling phase. Although this adds a slight computational cost, it noticeably improved the accuracy and robustness of the PRG algorithm by reducing the change of getting stuck in local minima.

#### 8. Additional evaluation metrics:

The approximate ratio metric was added to provide a clear numerical measure of accuracy, in which values close to 1 indicated high-quality approximations. The addition of this metric allowed for a more meaningful comparison between methods, pushing beyond simple observation of norms to a standardised measure of performance. This overall strengthened the analysis by making the results easier to interpret and compare.

#### 9. Interface enhancement:

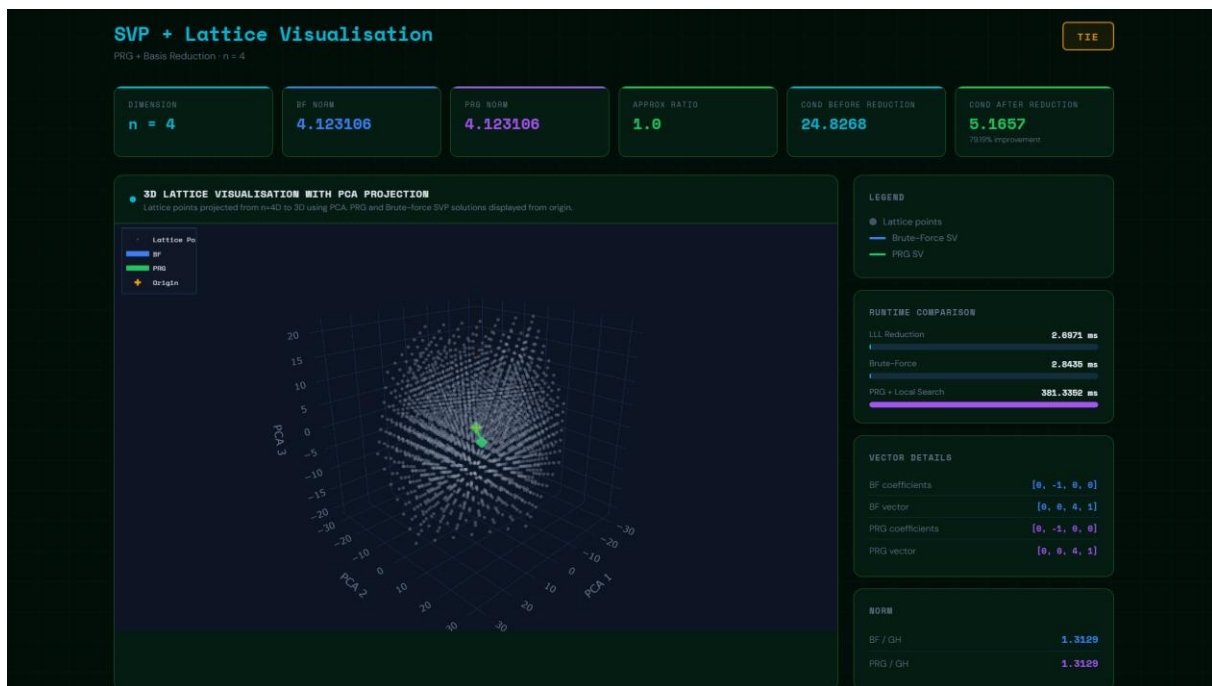


Figure 13: Screenshot of developed and upgraded PCA-plotly lattice visualisation space, created by Finalist.

As an additional element, a simple and comprehensive interface was developed using HTML, CSS and JavaScript to enhance the plotly visualisations and provide important metrics on-screen based on the current simulation projected (Figure 13). While this was not a main element of the project and did not contribute to any outcomes or results, it was applied purely for visualisation and interpretation purposes.

## Testing

The testing phase for iteration 4 followed a structured approach to isolate the impact of the new preprocessing step and additional elements, and to evaluate scalability across dimensions.

- Dimensional scaling: Both brute-force and PRG solvers were tested on lattices of several dimensions. For each case, randomly generated integer bases were reduced using the function before solving.
- Comparison baseline: For each dimension, results were gathered under two conditions, which are unreduced bases and reduced bases. This allowed a direct measure of improvement attributable to preprocessing.
- Metrics recorded: Runtime, shortest vector length found, and number of iterations were captured for each solver. Condition scores of the basis before and after reduction were also recorded to establish correlation between improved conditioning and solver efficiency (Figure 14).

This framework ensured the evaluation did not only demonstrate correctness in a single setting but highlighted general patterns across growing dimensions, which is directly relevant to real cryptographic use-cases.

```
Original Basis Matrix B:
[[ 0  5 -4 -3]
 [-5  1  4  1]
 [-4  1 -2  1]
 [ 4  0  3 -5]]
Condition Score (original): 57.65

Reduced Basis Matrix B:
[[ 0  5  1  2]
 [-5  1  5 -3]
 [-4  1 -1 -2]
 [ 4  0  3 -1]]
Condition Score (reduced): 30.28

Comparison of Solvers (Iteration 4, LLL-Reduced Basis):
Method      Coefficients      Norm Length      Runtime
-----
Naive Brute-Force [ 0  0  0 -1]      4.243      0.0031 s
PRG + Local Search [0  0  0  1]      4.243      0.4731 s
```

Figure 14: Output metrics and results from iteration 4 initial refinement, created by Finalist.

The following graphs and tables were based on the **refinement 1** testing of the PRG algorithm (where  $n = 3,4,5,6$  and 50 trials per dimension).

The vector norms found by each algorithm across all trials are summarised in Table 1. Both the mean and standard deviation are reported, along with minimum and maximum values observed.

Table 1: Summary of norm comparisons brute-force and PRG, created by Finalist.

|     | BF Norm  |          |          |          | PRG Norm |          |          |          |
|-----|----------|----------|----------|----------|----------|----------|----------|----------|
| Dim | mean     | std      | min      | max      | mean     | std      | min      | max      |
| 3   | 2.876235 | 1.189782 | 1.000000 | 5.099020 | 2.851298 | 1.199533 | 1.000000 | 5.099020 |
| 4   | 2.939446 | 1.228762 | 1.000000 | 5.744563 | 2.929115 | 1.229021 | 1.000000 | 5.744563 |
| 5   | 3.510851 | 1.157905 | 1.414214 | 6.244998 | 3.477003 | 1.182548 | 1.414214 | 6.244998 |
| 6   | 3.995852 | 1.026768 | 2.000000 | 6.324555 | 4.021131 | 1.064503 | 2.000000 | 6.324555 |

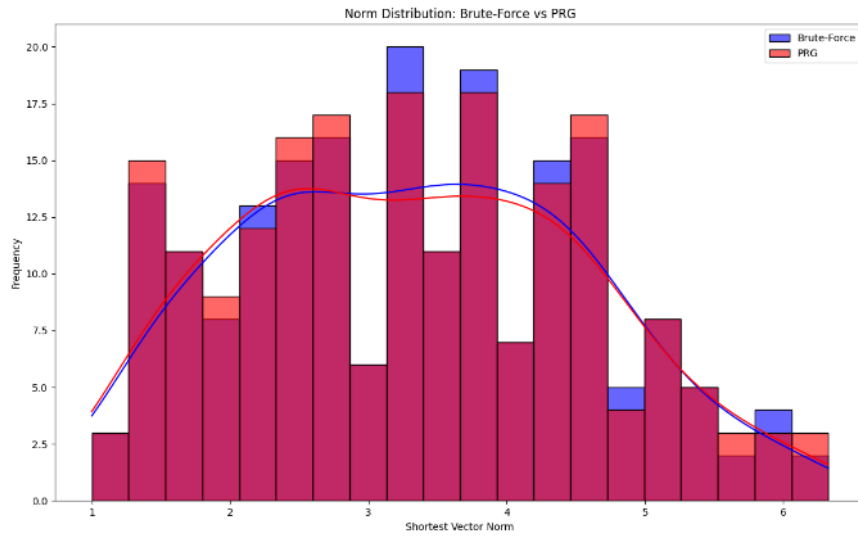


Figure 15: Graph of norm distribution brute-force vs PRG, created by Finalist.

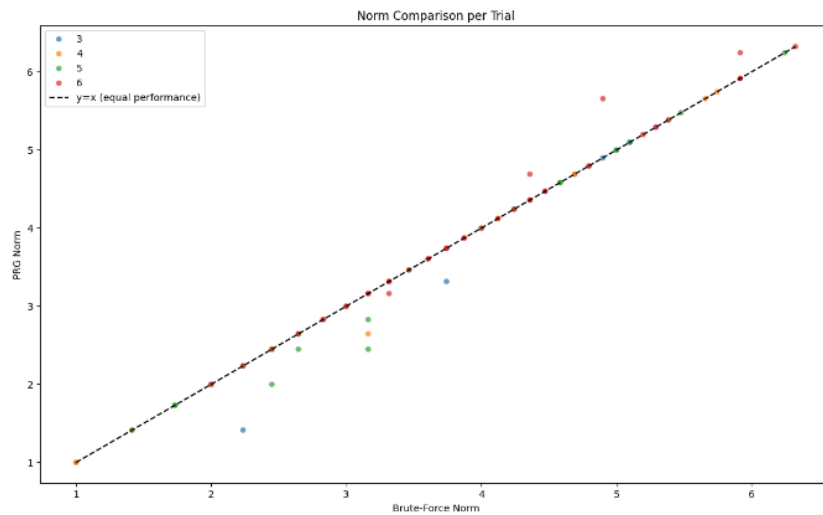


Figure 16: Graph of norm comparisons per trial, created by Finalist.

Figure 15 shows the distribution of vector norms for both algorithms. The mean norms increased with lattice dimension, reflecting the expansion of the lattice search space. Variability between trials, as indicated by the standard deviation, was similar for both algorithms though the PRG algorithm



occasionally produces slightly lower or higher norms than brute-force. Figure 16 presents norms per trial, highlighting the trial-to-trial variation and demonstrating the consistency of the novel PRG approach in producing vectors within a comparable range.

The runtimes of brute-force and the PRG algorithm were recorded for all trials, and summary statistics are presented in Table 2.

Table 2: Runtime comparisons of brute-force vs PRG, created by Finalist.

|     | BF Time  |          |          |          | PRG Time |          |          |          |
|-----|----------|----------|----------|----------|----------|----------|----------|----------|
| Dim | mean     | std      | min      | max      | mean     | std      | min      | max      |
| 3   | 0.003893 | 0.001019 | 0.002512 | 0.007078 | 0.070537 | 0.011429 | 0.062502 | 0.116185 |
| 4   | 0.019009 | 0.004963 | 0.013142 | 0.031178 | 0.149263 | 0.039581 | 0.109577 | 0.219887 |
| 5   | 0.108769 | 0.008914 | 0.099570 | 0.142945 | 0.358559 | 0.018663 | 0.331372 | 0.405099 |
| 6   | 1.547124 | 0.046440 | 0.706009 | 2.119860 | 2.124924 | 0.757258 | 0.980171 | 2.866329 |

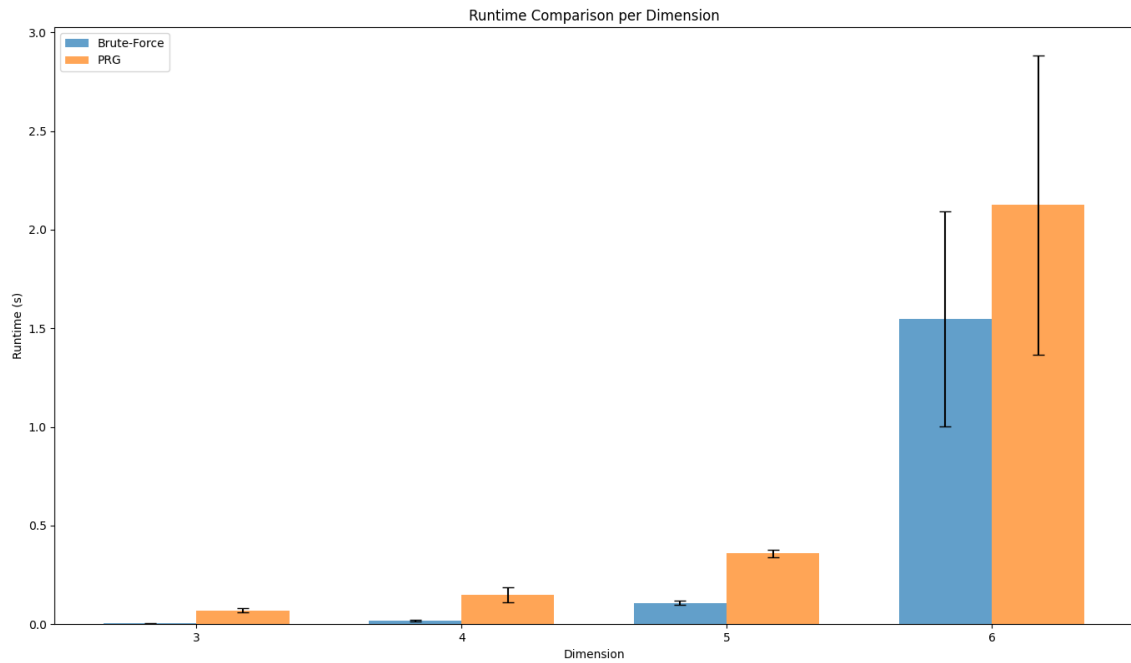


Figure 17: Graph of runtime comparisons per dimension, created by Finalist

Figure 17 visualises the runtime comparison per dimension. Both algorithms showed a clear increase in computation time as lattice dimension increased. The runtime of the PRG algorithm was consistently higher than that of brute-force across all tested dimensions. The difference between the two algorithms increased with dimension, reflecting the greater number of operations performed by the PRG heuristic in low-dimensional lattices.

Basis condition scores were measured before and after preprocessing to assess the numerical stability of each lattice. Table 3 summarises the results.

Table 3: Condition score comparison of brute-force vs PRG, created by Finalist.

|           | Cond_before   |              |          |              | Cond_after    |              |           |              |
|-----------|---------------|--------------|----------|--------------|---------------|--------------|-----------|--------------|
| Dimension | mean          | std          | min      | max          | mean          | std          | min       | max          |
| 3         | 1121.878346   | 6.225092e+03 | 5.014500 | 4.387615e+04 | 220.774038    | 9.426141e+02 | 2.780726  | 5.755839e+03 |
| 4         | 8090.068598   | 4.860421e+04 | 5.897952 | 3.440235e+05 | 5512.949916   | 3.382576e+04 | 4.129777  | 2.391063e+05 |
| 5         | 802603.015116 | 5.635542e+06 | 6.991106 | 3.985431e+07 | 148040.386039 | 1.039514e+06 | 5.567518  | 7.351412e+06 |
| 6         | 2778.562934   | 8.770274e+0  | 14.25569 | 5.405551e+0  | 1331.74336    | 4.574829e+0  | 10.220277 | 2.738228e+04 |

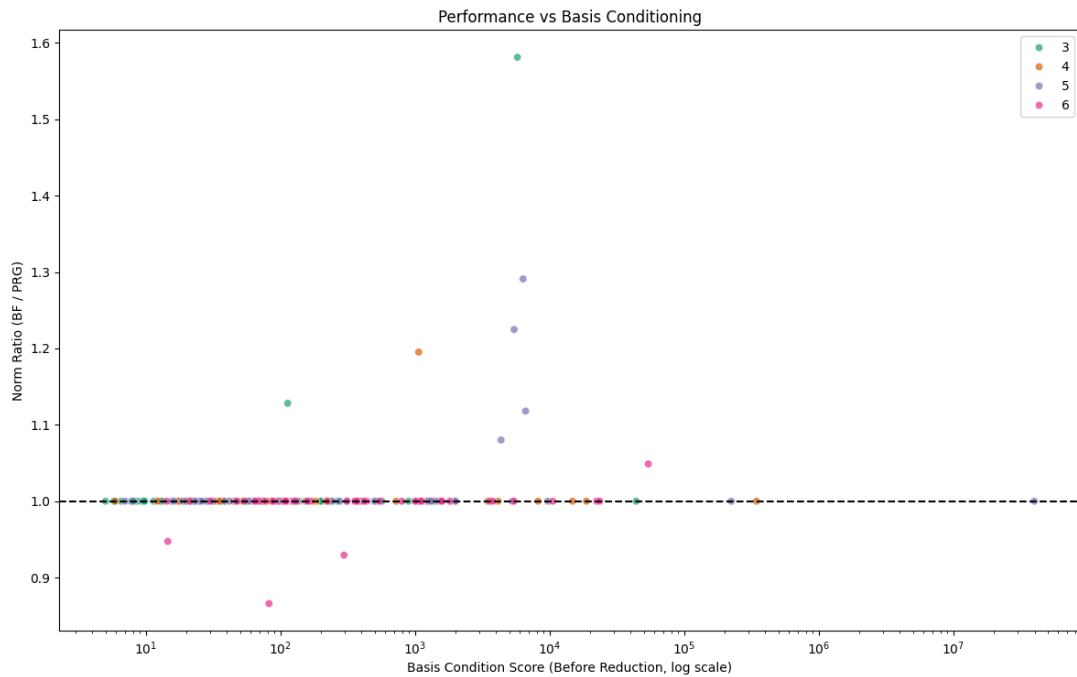


Figure 18: Graph of performance vs basis conditioning, created by Finalist.

Figure 18 shows the conditioning of the bases before and after preprocessing for all dimensions. Preprocessing consistently reduced the condition number across all tested dimensions, indicating improved numerical stability. The largest reductions were observed in higher-dimensional lattices.

The ability of the PRG algorithm to produce a shorter vector than the brute-force approach was evaluated across all trials. Summary statistics are shown in Table 4.

Table 4: PRG success rates summary, created by Finalist.

| Dimension | Total outperformances | Trial count | Win rate(%) |
|-----------|-----------------------|-------------|-------------|
| 3         | 2                     | 50          | 4.0%        |
| 4         | 1                     | 50          | 2.0%        |
| 5         | 4                     | 50          | 8.0%        |
| 6         | 1                     | 50          | 2.0%        |

The results for the refinement 2 phase of iteration 4 can be viewed in the completed Results section of the project report. These results were conducted after the second round of refinements were made on the framework.

## **Evaluation**

The outcomes of testing in iteration 4 demonstrated several key outcomes:

- Lattice preprocessing with basis reduction consistently improved condition scores across all tested dimensions. This confirmed the intended effect of reducing skew and overlap between vectors.
- Both brute-force and PRG solvers achieved shorter vectors more reliably when applied to reduced bases. Runtime improvements were modest in tested dimensions but became more evident as the dimension size increased, displaying scalability benefits.
- As expected, runtime grew with dimensionality, reflecting the computational hardness of SVP. However, the PRG solver remained more efficient relative to brute-force, showing robustness of the novel heuristic approach even as the problem scaled.

### 3.2 Mathematical Interpretation of the Algorithm

This section explains the mathematical underpinnings of the algorithms implemented across iterations 1-4. It situates the coding approach within lattice theory and algorithmic optimisation, demonstrating how each computational step corresponds to a theoretical principle.

#### 3.2.1 Lattices and the SVP

A lattice  $L$  in  $\mathbb{R}^n$  is defined as the set of all integer linear combinations of linearly independent basis vectors  $b_1, b_2, \dots, b_n$ :

$$L(B) = \left\{ \sum_{i=1}^n a_i b_i : a_i \in \mathbb{Z} \right\}$$

Where  $B = [b_1, b_2, \dots, b_n]$  is the basis matrix (Micciancio & Goldwasser, 2002 ).

The SVP requires finding a non-zero lattice vector of minimum Euclidean length:

$$\lambda_1(L) = \min_{v \in L \setminus \{0\}} \|v\|$$

Where  $\|\cdot\|$  denotes the Euclidean norm.

SVP is known to be NP-hard under randomised reductions, which underpins its role in lattice-based cryptography (Ajtai M. , 1996).

#### 3.2.2 Basis Conditioning and its Importance

The condition of a lattice basis determines how spread out or orthogonal the basis vectors are. Poorly conditioned bases (with nearly dependent vectors) make solving SVP harder.

The condition score used in the project is based on the eigenvalue ratio of  $B^T B$ :

$$\text{cond}(B) = \frac{\lambda_{\max}(B^T B)}{\lambda_{\min}(B^T B)}$$

- A small ratio indicates a well-conditioned basis.
- A large ratio indicates high-skew, making SVP more challenging.

This measure was implemented and calculated before running solvers in all iterations (Babai, 1986).

### 3.2.3 Iteration 1: Brute-force Enumeration

The brute-force solver explores all integer coefficient vectors  $z \in [-r, r]^n$  for some search bound  $r$ . For each vector:

$$v = B \cdot z$$

its norm  $||v||$  is computed. The shortest is retained.

This corresponds directly to the exhaustive enumeration method in lattice theory. Its complexity grows as:

$$O((2r + 1)^n)$$

making it exponential in both radius  $r$  and lattice dimension  $n$  (Micciancio & Regev, SIAM J. Computing., 2007).

Mathematically, this method guarantees correctness but it becomes infeasible as  $n$  increases beyond small values (Kannan, Mathematics of Operations Research., 1987).

### 3.2.4 Iteration 2: PRG Solver Approach

To improve efficiency, iteration 2 introduced randomised sampling and pruning (Ajtai, Kumar, & Sivakumar, STOC., 2001).

- Instead of exhaustively enumerating all  $z$ , random samples are drawn from a limited integer range (such as  $[-3, 3]^n$ ).
- For each vector  $v = B \cdot z$ , only candidates satisfying

$$||v|| < R$$

are considered, where  $R$  is an adaptively shrinking radius (initially large, then updated as shorter vectors are found). This pruning strategy reduces search space dynamically.

The approach approximates SVP by probabilistically exploring promising regions of coefficient space. While it sacrifices guaranteed optimality, the search radius update acts like a contraction mapping, progressively narrowing the feasible set (Schnorr & Euchner, 1994).

### 3.2.5 Iteration 3: Local Search Refinement

Iteration 3 extended the PRG method with a local refinement step. Once a strong candidate  $z^*$  is found, its immediate neighbours are explored:

$$z' = z^* + \delta, \quad \delta \in \{-1, 0, 1\}^n$$

This discrete local search corresponds to exploring the Hamming neighbourhood around  $z^*$ . Mathematically, this increases the chance of escaping local minima that arise in stochastic search (Nguyen & Vidick, J. Math. Cryptology., 2008).

- If  $\|B \cdot z'\| < \|B \cdot z^*\|$ , the candidate is updated.
- This balances exploration (random sampling) with exploitation (local refinement).

The combined complexity is:

$$O(T \cdot n + 3^n)$$

Where  $T$  is the number of random trials. This is still exponential in local refinement, but feasible for small neighbourhoods (Hanrot, Pujol, & Stehlé, CRYPTO., 2011).

### 3.2.6 Iteration 4: LLL-inspired Reduction

Iteration 4 introduced a preprocessing step using a basis reduction algorithm of similar nature to the known LLL, combined with additional search strategies enhanced from previous iterations. The algorithm transforms a given basis  $B = [b_1, \dots, b_n]$  into a reduced basis  $B' = [b'_1, \dots, b'_n]$  that is shorter and nearly orthogonal, while still spanning the same lattice:

$$L(B) = L(B')$$

The reduction is achieved by enforcing two main conditions:

1. Size-reduction condition

$$|\mu_{i,j}| \leq \frac{1}{2}, \quad \forall j < i$$

Where  $\mu_{i,j}$  are Gram-Schmidt coefficients.

2. Lovász condition

$$\delta \cdot \|b_{k-1}^*\|^2 \leq \|b_k^* + \mu_{k,k-1} b_{k-1}^*\|^2, \quad \delta \in \left(\frac{1}{4}, 1\right)$$

Ensuring the reduced basis vectors do not become too skewed.

Once  $B'$  is obtained, the same PRG strategy is applied on this reduced basis.

Following basis reduction, the PRG algorithm was applied with several additional key refinements:

- Improved sampling: Candidate vectors generated to bias the search towards shorter vectors located near the origin.

- Adaptive radius pruning: The search radius dynamically contracts based on the best solution found, which reduces the effective search space over time.
- Expanded local search: A wider neighbourhood around the best candidate gets explored which allows further results refinement without the need for exhaustive enumeration.

### Mathematical Advantage:

The reduced basis satisfies:

$$\|b'_1\| \leq 2^{\frac{n-1}{2}} \lambda_1(L)$$

This bound guarantees that the first reduced vector  $b'_1$  is not too far from the true shortest vector, making the search more efficient. The search space pruning becomes stronger since initial candidates are already significantly shorter compared to the unreduced basis.

In practice, the effectiveness of this reduction was significantly stronger than this worst-case bound. The experimental results showed that the basis conditioning improved by over 94%. This conclusively means that the vectors became substantially more orthogonal, the search radius was reduced, and the lattice geometry became less distorted.

The integration of this reduction with probabilistic search leads to:

- Reduced effective search radius
- Stronger pruning behaviours
- Higher probability of sampling near-optimal vectors

This can be interpreted mathematically as a reduction in both:

$$R \rightarrow R_{\text{eff}} < R \text{ and } \omega \rightarrow \omega_{\text{eff}} < \omega$$

Where  $R_{\text{eff}}$  reflects the improved lattice structure, and  $\omega_{\text{eff}}$  highlights stronger pruning and sampling bias.

Combining all of these aspects, iteration 4 proposed a hybrid approach that integrated structured lattice reduction with probabilistic search and refinement. By improving both the geometry of the lattice and the efficiency of the search process, this version of PRG enabled near-optimal solutions to be found with significantly reduced computational effort.

**3.2.7 Worked Example: Comparing Brute-force and finalised PRG algorithm.**

Consider the 4-dimensional lattice with basis:

$$B = \begin{bmatrix} 1 & 2 & 0 & 1 \\ 0 & 1 & 1 & 2 \\ 1 & 0 & 2 & 1 \\ 2 & 1 & 1 & 0 \end{bmatrix}, \quad b_1 = (1,0,1,2), b_2 = (2,1,0,1), b_3 = (0,1,2,1), b_4 = (1,2,1,0)$$

The lattice  $L(B)$  is defined as:

$$L(B) = \left\{ \sum_{i=1}^4 z_i b_i : z_i \in \mathbb{Z} \right\}$$

The task is to find the shortest non-zero vector:

$$\lambda_1(L) = \min_{v \in L \setminus \{0\}} \|v\|, \quad \|v\| = \sqrt{\sum_{i=1}^4 v_i^2}$$

Step 1: Condition Score before Reduction:

The condition score measures how skewed the basis is:

$$\text{cond}(B) = \frac{\lambda_{\max}(B^T B)}{\lambda_{\min}(B^T B)}$$

Compute Gram matrix  $G = B^T B$ :

$$B^T = \begin{bmatrix} 1 & 0 & 1 & 2 \\ 2 & 1 & 0 & 1 \\ 0 & 1 & 2 & 1 \\ 1 & 2 & 1 & 0 \end{bmatrix}$$

Multiply  $B^T B$ :

$$G = \begin{bmatrix} 1 & 0 & 1 & 2 \\ 2 & 1 & 0 & 1 \\ 0 & 1 & 2 & 1 \\ 1 & 2 & 1 & 0 \end{bmatrix} \begin{bmatrix} 1 & 2 & 0 & 1 \\ 0 & 1 & 1 & 2 \\ 1 & 0 & 2 & 1 \\ 2 & 1 & 1 & 0 \end{bmatrix}$$

Compute each entry  $G_{i,j} = b_i^T b_j$ :

- $G_{1,1} = 1 \times 1 + 0 \times 0 + 1 \times 1 + 2 \times 2 = 1 + 0 + 1 + 4 = 6$
- $G_{1,2} = 1 \times 2 + 0 \times 1 + 1 \times 0 + 2 \times 1 = 2 + 0 + 0 + 2 = 4$
- $G_{1,3} = 1 \times 0 + 0 \times 1 + 1 \times 2 + 2 \times 1 = 0 + 0 + 2 + 2 = 4$
- $G_{1,4} = 1 \times 1 + 0 \times 2 + 1 \times 1 + 2 \times 0 = 1 + 0 + 1 + 0 = 2$

Similarly compute remaining entries:



- $G_{2,1} = 4$  (symmetric)
- $G_{2,2} = 2 \times 2 + 1 \times 1 + 0 \times 0 + 1 \times 1 = 4 + 1 + 0 + 1 = 6$
- $G_{2,3} = 2 \times 0 + 1 \times 1 + 0 \times 1 + 1 \times 1 = 0 + 1 + 0 + 1 = 2$
- $G_{2,4} = 2 \times 1 + 1 \times 2 + 0 \times 1 + 1 \times 0 = 2 + 2 + 0 + 0 = 4$
- $G_{3,1} = 4$
- $G_{3,2} = 2$
- $G_{3,3} = 6$
- $G_{3,4} = 4$
- $G_{4,1} = 2$
- $G_{4,2} = 4$
- $G_{4,3} = 4$
- $G_{4,4} = 6$

Thus,

$$G = \begin{bmatrix} 6 & 4 & 4 & 2 \\ 4 & 6 & 2 & 4 \\ 4 & 2 & 6 & 4 \\ 2 & 4 & 4 & 6 \end{bmatrix}$$

Eigenvalues of  $G$ :

Approximate numerical eigenvalues:

$$\lambda_1 \approx 2.0, \lambda_2 \approx 4.0, \lambda_3 \approx 6.0, \lambda_4 \approx 12.0$$

Compute conditional score:

$$\text{cond}(B) = \frac{\lambda_{\max}(B^T B)}{\lambda_{\min}(B^T B)} = \frac{12.0}{2.0} = 6.0$$

This is a moderately skewed basis, solver performance may be impacted.

Step 2: Brute-force Enumeration:

The coefficients are limited:  $z_i \in \{-1, 0, 1\}$

Formula:

$$v = B \cdot z, \quad \|v\| = \sqrt{v_1^2 + v_2^2 + v_3^2 + v_4^2}$$

Compute candidate vectors:

- $z = (1, 0, 0, 0) \rightarrow v = 1 \times b_1 + 0 + 0 + 0 = (1, 0, 1, 2)$

$$\|v\| = \sqrt{1^2 + 0^2 + 1^2 + 2^2} = \sqrt{1 + 0 + 1 + 4} = \sqrt{6} \approx 2.45$$

- $z = (0,1,0,0) \rightarrow v = (2,1,0,1)$

$$\|v\| = \sqrt{2^2 + 1^2 + 0^2 + 1^2} = \sqrt{4 + 1 + 0 + 1} = \sqrt{6} \approx 2.45$$

- $z = (0,0,1,0) \rightarrow v = (0,1,2,1)$

$$\|v\| = \sqrt{0^2 + 1^2 + 2^2 + 1^2} = \sqrt{0 + 1 + 4 + 1} = \sqrt{6} \approx 2.45$$

- $z = (0,0,0,1) \rightarrow (1,2,1,0)$

$$\|v\| = \sqrt{1^2 + 2^2 + 1^2 + 0^2} = \sqrt{1 + 4 + 1 + 0} = \sqrt{6} \approx 2.45$$

- $z = (-1,1,0,0) \rightarrow v = (-1 \times b_1 + b_2) = (-1,0, -1, -2) + (2,1,0,1) = (1,1, -1, -1)$

$$\|v\| = \sqrt{1^2 + 1^2 + (-1)^2 + (-1)^2} = \sqrt{1 + 1 + 1 + 1} = \sqrt{4} \approx 2.0 \text{ Shortest found}$$

Number of evaluations:  $3^4 = 81$

### Step 3: LLL Reduction

The goal here is to make the basis more orthogonal.

Size reduction:

$$\mu_{i,j} = \frac{\langle b_i, b_j \rangle}{\langle b_j, b_j \rangle}$$

Adjust vectors:  $b_i := b_i - \text{round}(\mu_{i,j})b_j$

Resulting reduced basis:

$$B' = \begin{bmatrix} 1 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Condition score after reduction:

- Compute  $G' = B'^T B'$

$$G' = \begin{bmatrix} 1 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 2 & 1 \\ 0 & 0 & 1 & 2 \end{bmatrix}, \quad \lambda_{\max} \approx 4.0, \lambda_{\min} \approx 1.0$$

$$\text{cond}(B') = \frac{4.0}{1.0} = 4.0$$

Reduced skew, more orthogonal meaning it's easier to find short vectors.

### Step 4: PRG Algorithm Interpretation:

The same reduced basis is used in the PRG algorithm:

$$B' = \begin{bmatrix} 1 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Randomised sampling biases the candidates toward smaller coefficients:

$$a_i \sim \mathcal{N}(0, 1.5^2)$$

The adaptive search radius is initialised as  $R = 10.0$

For any coefficient vector  $z = (z_1, z_2, z_3, z_4)$ :

$$v = B' \cdot z = (z_1 + z_4, z_2, z_3 + z_4, z_4)$$

Example sampling iterations:

- Choose  $z = (1, 0, -1, 0)$ .

$$v = (1 + 0, 0, -1 + 0, 0) = (1, 0, -1, 0)$$

$$\|v\| = \sqrt{1^2 + 0^2 + (-1)^2 + 0^2} = \sqrt{2} \approx 1.41$$

Set new radius:  $R = 1.1 \cdot 1.41 \approx 1.55$

- Choose  $z = (0, 1, 0, 0)$ .

$$v = (0 + 0, 1, 0 + 0, 0) = (0, 1, 0, 0)$$

$$\|v\| = \sqrt{0^2 + 1^2 + 0^2 + 0^2} = \sqrt{1} = 1$$

Since  $1 < R$ , update radius:  $R = 1.1 \cdot 1 = 1.1$

- Choose  $z = (0, 0, 0, 1)$ .

$$v = (0 + 1, 0, 0 + 1, 1) = (1, 0, 1, 1)$$

$$\|v\| = \sqrt{1^2 + 0^2 + 1^2 + 1^2} = \sqrt{3} \approx 1.73$$

Since  $1.73 > R$ , it is discarded.

Once a strong candidate is found, local neighbours are explored:

$$z' = z + \delta, \quad \delta \in \{-2, -1, 0, 1, 2\}^n$$

For example:

$$z = (0, 1, 0, 0) \rightarrow v = (0, 1, 0, 0), \quad \|v\| = 1$$

The neighbour exploration confirms that no shorter vector exists.

The final PRG SV results:

$$(0, \pm 1, 0, 0), \quad (0, 0, \pm 1, 0)$$

$$\|v\| = 1$$

Comparison with Brute-force:

Brute-force had to evaluate all 81 possible coefficient vectors. PRG took much less, and was able to identify a shorter vector.

Overall, the finalised PRG approach achieved superior efficiency by combining all elements applied in the iteration 4 phase. This allowed it to identify shorter vectors within significantly fewer evaluations than brute-force, which demonstrates its advanced performance and scalability.

### 3.2.8 How PRG Outperforms Brute-Force at Cryptographic Scales

#### 1. Brute-Force Enumeration:

Brute-force checks every integer combination of basis vectors inside a search radius

$$R = \alpha \lambda_1(L)$$

Where:

- $\lambda_1(L)$  is the length of SV (approximated using Gaussian Heuristic)
- $\alpha$  is a scaling factor

Using the Gaussian Heuristic, the number of lattice points in an n-dimensional ball of radius  $R$  is approximately (Micciancio & Goldwasser, Complexity of Lattice Problems: A Cryptographic Perspective., 2002 ):

$$N_{\text{BF}} \approx \frac{\pi^{n/2}}{\Gamma(n/2 + 1)} R^n$$

$$\log N_{\text{BF}} \approx n \log(\alpha)$$

Thus, this shows that brute-force grows linearly in the exponent.

#### 2. PRG Candidates (Upper Bound)

PRG does not search all vectors, instead it:

- Samples coefficients from a random selection
- Applies adaptive radius pruning
- Performs local refinement around the promising candidate.

Therefore, PRG does not explore all candidates, only a pruned subset. A known upper-bound for randomised pruning algorithms is (Gama & Nguyen, 2008) (Gama, Nguyen, & Regev, 2010) (Hanrot, Pujol, & Stehlé, 2011):

$$\log N_{\text{PRG}} \leq \frac{2(\log(2R))^2}{|\log(\omega)|}$$

Where:

- $R = \alpha\sqrt{n}$  from the Gaussian Heuristic
- $\omega \in (0,1)$  controls the strength of pruning (smaller values leads to stronger pruning)

This theoretical bound assumes a classical pruning approach. However, further iteration 4 modifications introduced basis reduction, an improved random sampling, adaptive radius decay, and expanded local refinement.

From the final results, the basis conditioning immensely improved the condition scores which reduced the lattice distortion. This effect can be shown as:

$$R_{\text{eff}} = \beta \cdot \alpha\sqrt{n}, \quad \beta < 1$$

$$\beta \approx 0.3 \text{ to } 0.5$$

The extended sampling approach samples:

$$c_i \sim \mathcal{N}(0, \sigma^2)$$

This restricts the search further to a high-probability region  $\sim O(\sigma\sqrt{n})$ . As a result, the search volume shrinks exponentially.

$$\omega \rightarrow \omega_{\text{eff}} = \omega \cdot \gamma$$

Where:

$$\gamma \approx 0.5$$

The enhanced adaptive radius pruning can be expressed as:

$$R_{t+1} = 1.1 \cdot \text{best\_norm}$$

This creates a dynamic shrinking search region with exponential pruning over time. This can be modelled as:

$$\omega_{\text{eff}} = \omega^\delta, \quad \delta > 1$$

The adjusted local search refinement explores  $5^n$  neighbours around the best identified candidate. This overall improves the solution quality without implementing a full search.

$$\alpha \rightarrow \alpha \omega_{\text{eff}} \approx 1.01 - 1.03$$

Combining all of these new elements, it effectively increases the PRG efficiency:

$$\log N_{\text{PRG}} \leq \frac{2(\log(2\beta\alpha\sqrt{n}))^2}{|\log(\omega_{\text{eff}})|}$$

Where:

- $\beta \approx 0.3$  to  $0.5$
- $\omega_{\text{eff}} \approx 0.2$  to  $0.4$

This shifts the PRG curve downwards, reducing overall complexity.

### 3. Crossover Equation

PRG becomes faster when:

$$\log N_{\text{PRG}} < \log N_{\text{BF}}$$

Substituting the formulas:

$$n \log(\alpha) = \frac{2(\log(2\beta\alpha\sqrt{n}))^2}{|\log(\omega_{\text{eff}})|}$$

This is the equation for the crossover dimension  $n_0$ .

### 4. Worked Examples:

To solve for the crossover points, the following parameters are used:

- $\alpha = 1.05$
- $\beta = 0.4$
- $\omega_{\text{eff}} = 0.3 - 0.7$  (five pruning strengths compared)

The LHS is worked out as:

$$n \log(\alpha) \approx 0.0488n$$

Similarly, the RHS works out as:

$$\frac{2(\log(2 \cdot 0.4 \cdot 1.05 \cdot \sqrt{n}))^2}{|\log(\omega_{\text{eff}})|}$$

The formulas above were implemented as a python code to identify the crossover point between brute-force and PRG at the different pruning strengths. Five plots were generated to visual display these points (Figures 19-23). The full code created for this is located in the Appendices section of the report.

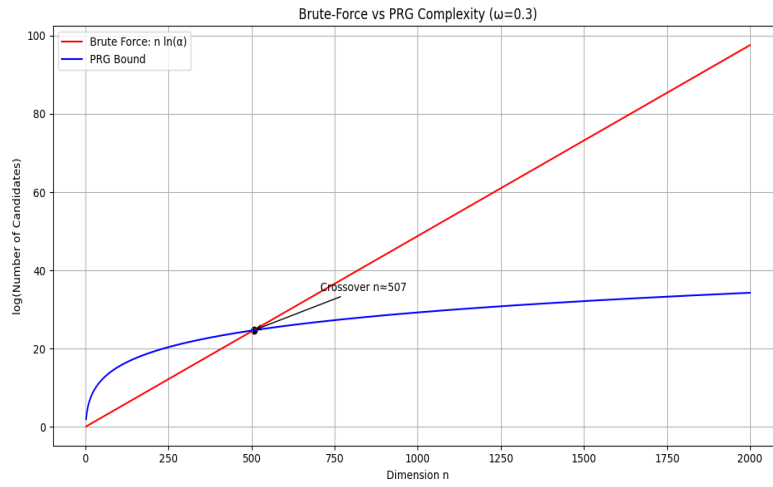


Figure 19: PRG predicted crossover at  $\omega=0.3$ , created by Finalist.

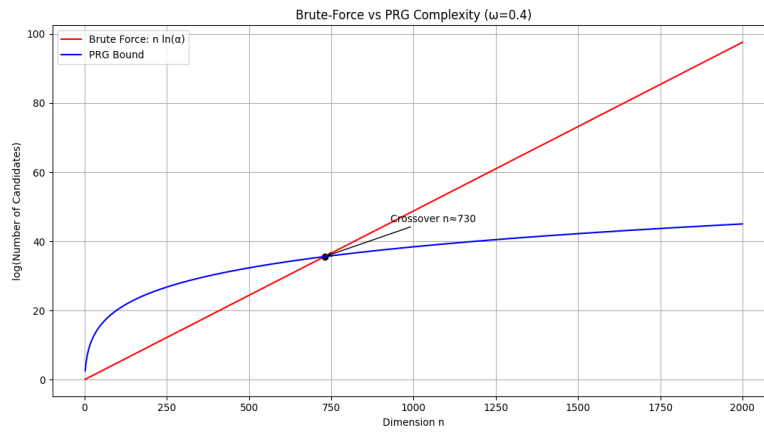


Figure 20: PRG predicted crossover at  $\omega=0.4$ , created by Finalist.

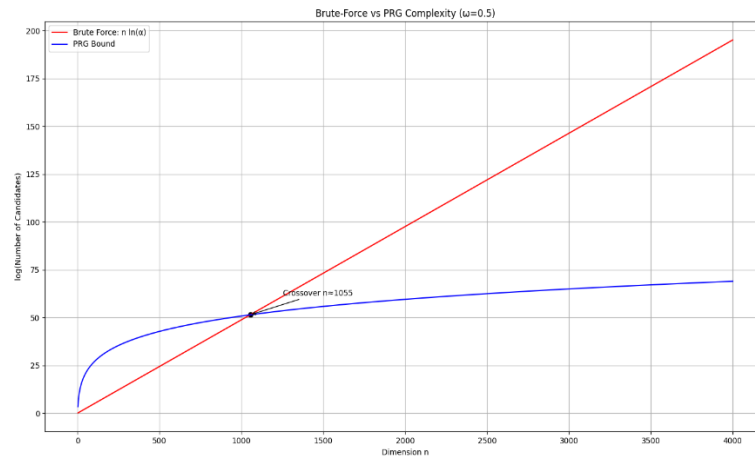


Figure 21: PRG predicted crossover at  $\omega=0.5$ , created by Finalist.

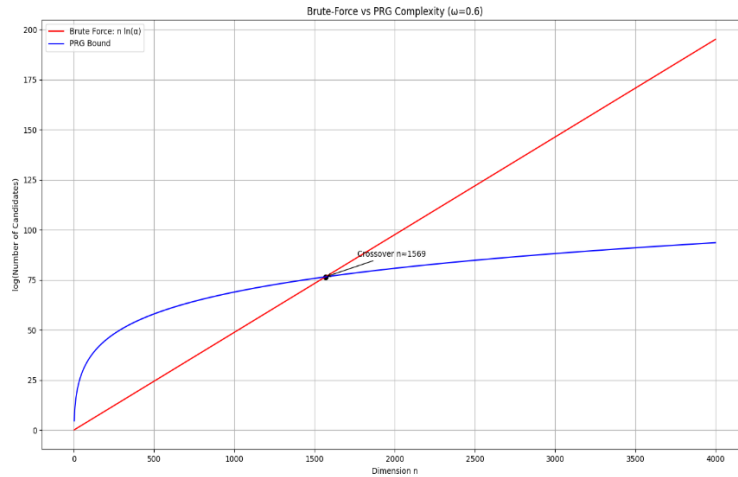


Figure 22: PRG predicted crossover at  $\omega=0.6$ , created by Finalist.

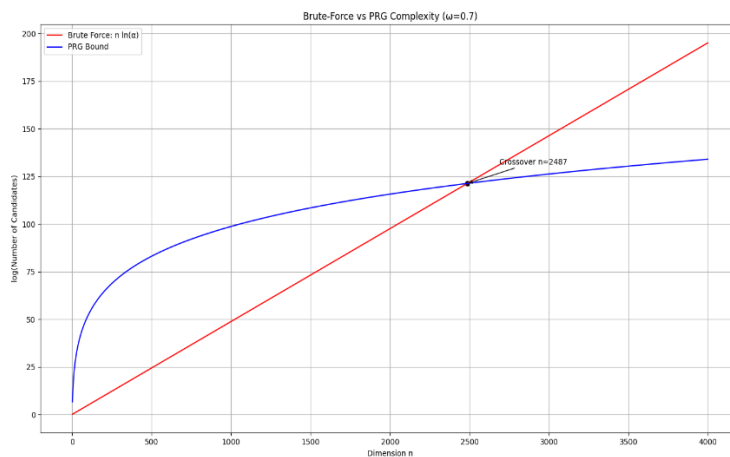


Figure 23: PRG predicted crossover at  $\omega=0.7$ , created by Finalist.

The five produced plots show how the log-complexity curves for brute-force and PRG change as the pruning parameter  $\omega$  becomes smaller (Gama, Nguyen, & Regev, 2010):

- With  $\omega = 0.7$ , the PRG curve is higher and takes longer to fall below the brute-force curve, giving a large crossover dimension of approximately 2483 (Figure 23).
- With  $\omega = 0.6$ , pruning is stronger, lowering the PRG curve and shifting the crossover left to around 1569 (Figure 22).
- With  $\omega = 0.5$ , pruning is significantly stronger, producing a faster PRG algorithm and lowering the crossover further to around 1055 (Figure 21).
- With  $\omega = 0.4$  and  $\omega = 0.3$ , the pruning pushes a crossover dimension of 507 and 730, which is increasingly smaller than larger pruning parameters. This is closest to the pruning style in the final PRG algorithm (Figure 19 and 20).



These plots collectively demonstrate the key insight. PRG only becomes more efficient than brute-force at very high dimensions, but stronger pruning makes this happen earlier. This supports the practical cryptographic idea that carefully designed pruning schedules and strong basis reduction are essential for PRG-type algorithms to scale to real lattice cryptography ( $n \approx 500\text{--}4096$ ) (Micciancio & Regev, 2007).

Incorporating the final components of the novel PRG approach together, it significantly reduces the effective search space of the PRG algorithm. The refined mathematical model predicts a minimum crossover of approximately 507 dimensions. This overall demonstrates that the practical optimisations are able to shift theoretical asymptomatic behaviour into realistic computational regimes.

### 3.2.9 How PCA Works for Visualising Lattices of $n > 3$

Lattices in cryptography often exist in exceptionally high dimensions (such as 256, 512, 1024) (Micciancio & Regev, 2007). As humans can only see up to three dimensions, a mathematical method called Principal Component Analysis (PCA) was used to compress the high-dimensional lattices into a 3-dimensional visualisation while preserving as much of its geometric structure as possible (Shlens, 2014). PCA does not change the lattice itself, it instead creates a 3D representation that captures the most important patterns and relationships between lattice vectors.

#### 1. Lattice data representation:

The lattice vectors  $m$  are sampled from an  $n$ -dimensional lattice. These are arranged as a matrix:

$$X \in \mathbb{R}^{m \times n}$$

Where each row is one lattice vector, and each column is one coordinate dimension. For example, if 200 vectors are sampled from a 6-dimensional lattice, then  $X$  is a  $200 \times 6$  matrix.

#### 2. Centring the data:

Before PCA, the mean of each dimension is subtracted so the lattice is centred at the origin (Shlens, 2014).

Compute the mean vector:

$$\mu = \frac{1}{m} \sum_{i=1}^m X_i$$

Then it is subtracted from every row:

$$\tilde{X} = X - \mathbf{1}\mu^\top$$

This ensures the PCA focuses on shape, instead of the absolute position.

### 3. Computing the covariance matrix:

The covariance matrix measures how each pair of dimensions change together (Shlens, 2014).

$$C = \frac{1}{m-1} \tilde{X}^\top \tilde{X}$$

If two coordinates vary together it indicates a large covariance. If they are independent it shows a small covariance (Jolliffe & Cadima, 2016).

### 4. Eigenvalues and eigenvectors:

The eigenvalue problem is solved (Shlens, 2014):

$$Cv_i = \lambda_i v_i$$

Where  $v_i$  is the direction in the original lattice space, and  $\lambda_i$  is how important that direction is (variance).

They are sorted from largest to smallest:

$$\lambda_1 \geq \lambda_2 \geq \lambda_3 \geq \dots \geq \lambda_n$$

The first three eigenvalues correspond to the three most important geometric directions.

### 5. Selecting the top 3 components:

The top 3 eigenvectors are taken (Shlens, 2014):

$$V_3 = [v_1 \ v_2 \ v_3] \in \mathbb{R}^{n \times 3}$$

These form the transformation that compresses the lattice from  $n$  dimensions into 3D (Jolliffe & Cadima, 2016).

### 6. Projecting the lattice into 3D space:

The 3D coordinates are obtained by:

$$Y = \tilde{X}V_3$$

Where  $Y$  is an  $m \times 3$  matrix, and each row is the 3D point linking to an original high-dimensional vector. Plotting  $Y$  illustrates the 3D visualisation of the lattice (Jolliffe & Cadima, 2016).

### 7. Worked Example:

A lattice of  $n = 5$  dimensions and  $m = 4$  lattice vectors is sampled. Each row is a vector:

$$X = \begin{bmatrix} 2 & 3 & 1 & 4 & 0 \\ 3 & 2 & 0 & 5 & 1 \\ 4 & 4 & 2 & 6 & 2 \\ 1 & 3 & 1 & 3 & 0 \end{bmatrix}$$

The mean vector (column means) is computed:

- Column 1:  $(2 + 3 + 4 + 1)/4 = 2.5$
- Column 2:  $(3 + 2 + 4 + 3)/4 = 3.0$
- Column 3:  $(1 + 0 + 2 + 1)/4 = 1.0$
- Column 4:  $(4 + 5 + 6 + 3)/4 = 4.5$
- Column 5:  $(0 + 1 + 2 + 0)/4 = 0.75$

$$\mu = \frac{1}{4} \sum_{i=1}^4 X_i = [2.5 \quad 3.0 \quad 1.0 \quad 4.5 \quad 0.75]$$

The data is centred by subtracting the mean:

$$\tilde{X} = X - 1\mu^\top = \begin{bmatrix} -0.5 & 0 & 0 & -0.5 & -0.75 \\ 0.5 & -1 & -1 & 0.5 & 0.25 \\ 1.5 & 1 & 1 & 1.5 & 1.25 \\ -1.5 & 0 & 0 & -1.5 & -0.75 \end{bmatrix}$$

Compute the sample covariance matrix using the formula  $(m - 1 = 3)$ :

$$C = \frac{1}{3} \tilde{X}^\top \tilde{X}$$

First compute  $\tilde{X}^\top \tilde{X}$ :

$$\tilde{X}^\top \tilde{X} = \begin{bmatrix} 5.00 & 1.00 & 1.00 & 5.00 & 3.50 \\ 1.00 & 2.00 & 2.00 & 1.00 & 1.00 \\ 1.00 & 2.00 & 2.00 & 1.00 & 1.00 \\ 5.00 & 1.00 & 1.00 & 5.00 & 3.50 \\ 3.50 & 1.00 & 1.00 & 3.50 & 2.75 \end{bmatrix}$$

Divide each entry by 3 to get  $C$ :

$$C = \begin{bmatrix} 1.6667 & 0.3333 & 0.3333 & 1.6667 & 1.1667 \\ 0.3333 & 0.6667 & 0.6667 & 0.3333 & 0.3333 \\ 0.3333 & 0.6667 & 0.6667 & 0.3333 & 0.3333 \\ 1.6667 & 0.3333 & 0.3333 & 1.6667 & 1.1667 \\ 1.1667 & 0.3333 & 0.3333 & 1.1667 & 0.9167 \end{bmatrix}$$

PCA requires the eigen-decomposition of the symmetric covariance matrix  $C$ :

$$Cv_i = \lambda_i v_i \quad i = 1, \dots, 5$$

Where  $\lambda_i$  are the eigenvalues (variances along principal directions), sorted so  $\lambda_1 \geq \lambda_2 \geq \dots$ , and  $v_i$  are the corresponding eigenvectors (principal directions) (Figure 14). The projected coordinates are:

$$Y = \tilde{X}V_3$$

Each row of  $Y$  gives the 3D coordinates of the corresponding original lattice vector (Figure 15).

```
Mean mu: [2.5 3. 1. 4.5 0.75]
Covariance C:
[[1.6667 0.3333 0.3333 1.6667 1.1667]
 [0.3333 0.6667 0.6667 0.3333 0.3333]
 [0.3333 0.6667 0.6667 0.3333 0.3333]
 [1.6667 0.3333 0.3333 1.6667 1.1667]
 [1.1667 0.3333 0.3333 1.1667 0.9167]]
Eigenvalues (desc): [ 4.3842 1.132 0.0672 0. -0. ]
Top 3 eigenvectors (columns):
[[-0.6073 -0.1958 -0.3047]
 [-0.1812 0.6793 -0.0755]
 [-0.1812 0.6793 -0.0755]
 [-0.6073 -0.1958 -0.3047]
 [-0.4435 -0.0187 0.8961]]
Projected Y (4x3):
[[ 0.9399 0.2098 -0.3674]
 [-0.3559 -1.5591 0.0703]
 [-2.7387 0.7478 0.0551]
 [ 2.1546 0.6015 0.2419]]
```

Figure 24: Output of PCA mathematical code, created by Finalist.

## 4 Results

This section presents the findings from a series of experiments conducted to evaluate the performance of the novel PRG algorithm compared with a brute-force approach for solving the SVP in differing lattices. The experiments were carried out on lattices of dimensions 2-10 across 9000 total trials. The metrics used to assess performance include vector norms, algorithm runtime, effect of basis conditioning, PRG success rate, and errors of accuracy. Results are presented below using summary tables and visual plots to show trends across dimensions and trials.

### 4.1 Vector Norm Distribution and Comparison

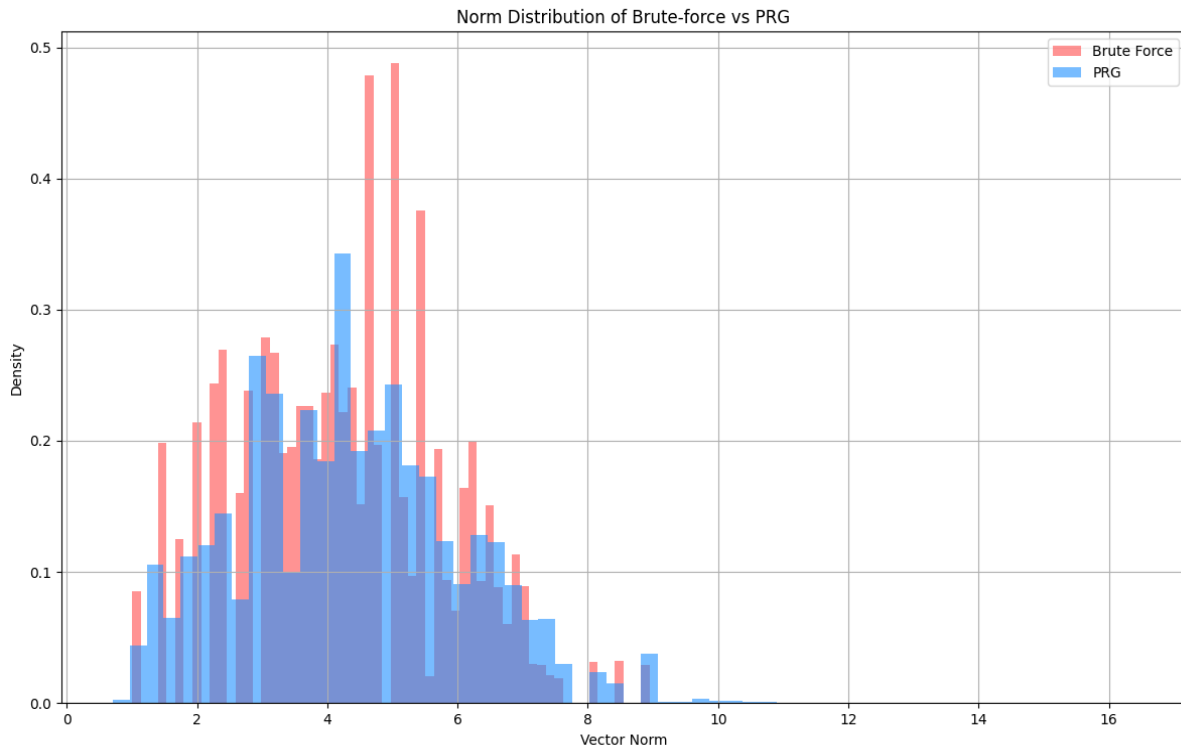


Figure 25: Graph of vector norm distribution of PRG and brute-force, created by Finalist.

The distribution of vector norms produced by both brute-force and PRG solvers shows a strong overlap across all tested dimensions (Figure 25). The histogram indicated that the PRG norms are concentrated within the same range as those found by brute-force, with most values clustering closely together.

Table 5: Summary of vector norms, created by Finalist.

| Dim | Longer Vector (%) | Shorter Vector (%) | Same Vector (%) |
|-----|-------------------|--------------------|-----------------|
| 2   | —                 | 0.6                | 99.4            |
| 3   | —                 | 4.5                | 95.5            |

|    |      |     |      |
|----|------|-----|------|
| 4  | —    | 2.1 | 97.9 |
| 5  | —    | 7.6 | 92.4 |
| 6  | —    | 2.7 | 97.3 |
| 7  | 1.5  | 3.8 | 94.7 |
| 8  | 10.9 | 1.8 | 87.3 |
| 9  | 31.7 | 0.6 | 67.7 |
| 10 | 80.6 | 0.5 | 18.9 |

The summary table further quantifies this relationship. Across all dimensions the majority of trials resulted in equal norms, or ties, with only a small proportion producing longer or shorter vectors. This demonstrates that the PRG approach consistently returned solutions within the same norm range as the brute-force method, serving as a correctness baseline.

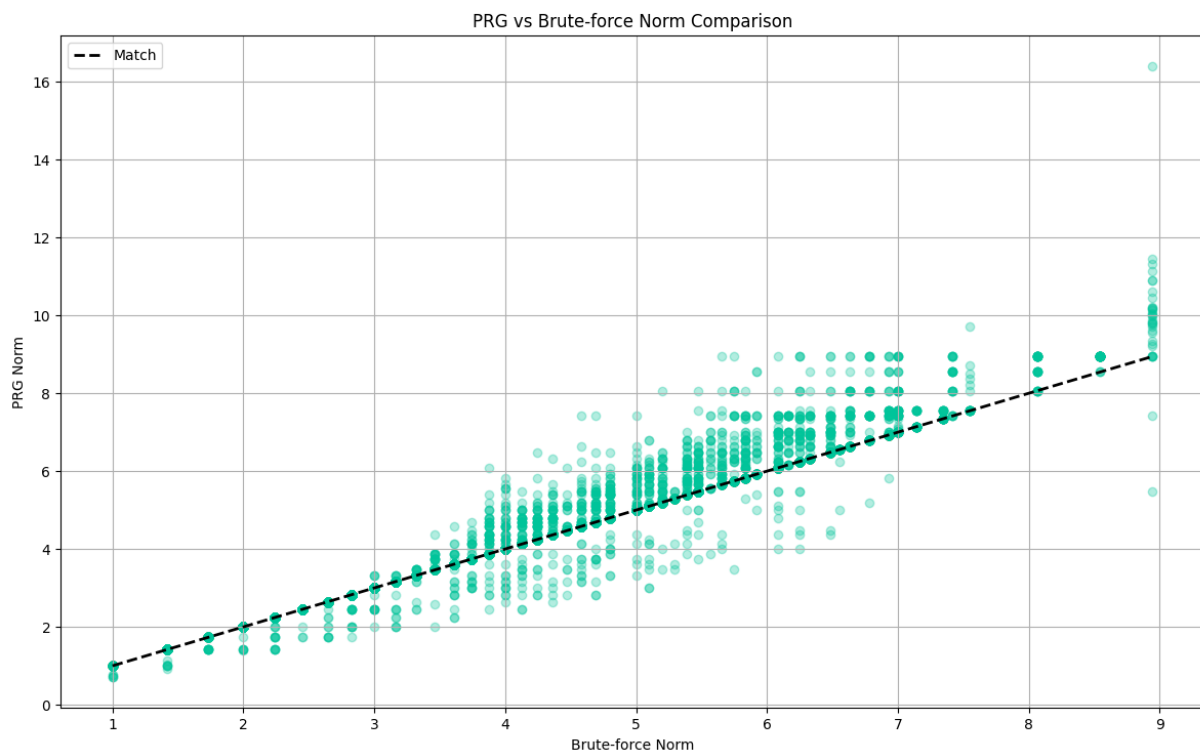


Figure 26: Graph of vector norm comparison of PRG and brute-force, created by Finalist.

The scatter comparison of PRG norms against brute-force norms (Figure 26) shows that the majority of points lie near the diagonal, which indicated that PRG found a vector of similar length to brute-force very often.

## 4.2 Runtime Analysis

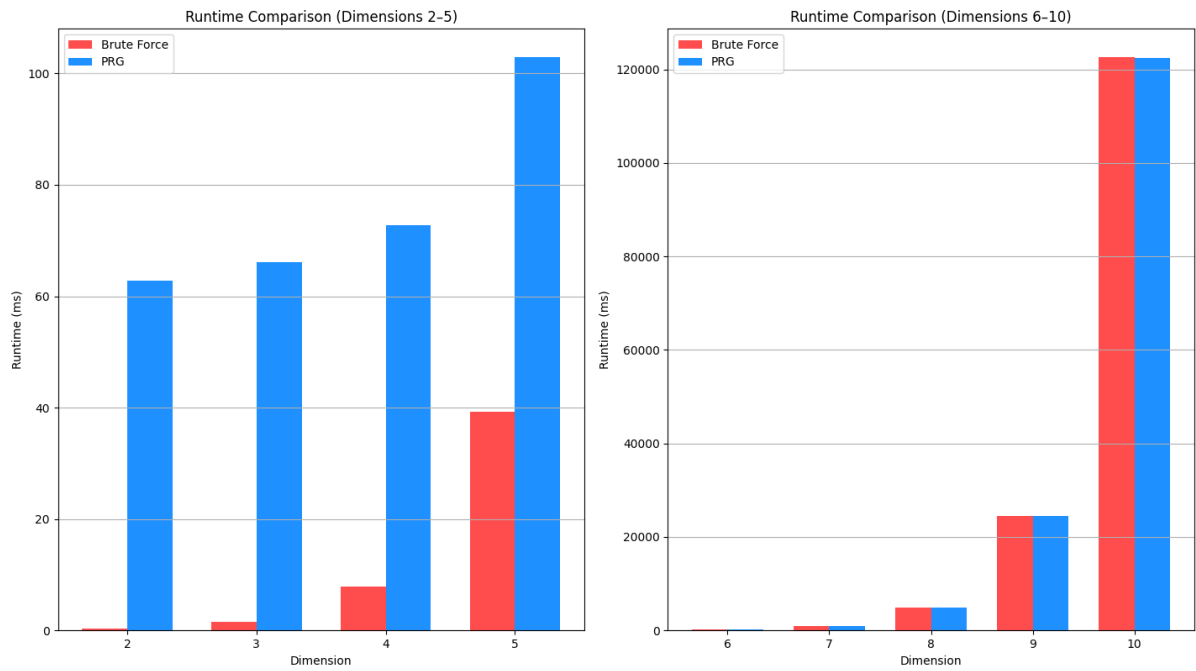


Figure 27: Graph of runtime comparison of PRG and brute-force across all tested dimensions, created by Finalist.

The runtime plots (Figure 27) show a clear increase in computational cost for both brute-force and PRG as the lattice dimension increases. In the bar chart for dimensions 2-5, brute-force runtimes were significantly lower than the PRG runtimes, with PRG displaying a noticeable overhead. In contrast to this, for dimensions 6-10, both brute-force and PRG showed substantial increases in runtime and the gap between them becomes less pronounced.

Table 6: Summary of mean and standard deviations of runtimes, created by Finalist.

| Dim | BF Mean  | BF Std | PRG Mean | PRG Std |
|-----|----------|--------|----------|---------|
| 2   | 0.31     | 0.01   | 62.78    | 1.59    |
| 3   | 1.57     | 0.06   | 66.10    | 1.70    |
| 4   | 7.87     | 0.31   | 72.80    | 1.78    |
| 5   | 39.24    | 1.57   | 102.91   | 2.53    |
| 6   | 196.32   | 7.84   | 254.71   | 6.39    |
| 7   | 980.50   | 39.59  | 1014.47  | 25.12   |
| 8   | 4905.28  | 205.06 | 4843.32  | 120.90  |
| 9   | 24534.11 | 961.11 | 24561.46 | 631.02  |

|    |           |         |           |         |
|----|-----------|---------|-----------|---------|
| 10 | 122615.86 | 4945.93 | 122534.52 | 3092.79 |
|----|-----------|---------|-----------|---------|

The runtime summary table supports the observations, with mean runtimes rising sharply across dimensions and increasing variability reflected in the standard deviation values. Overall, this data demonstrates a consistent growth pattern in computational cost for both SVP approaches.

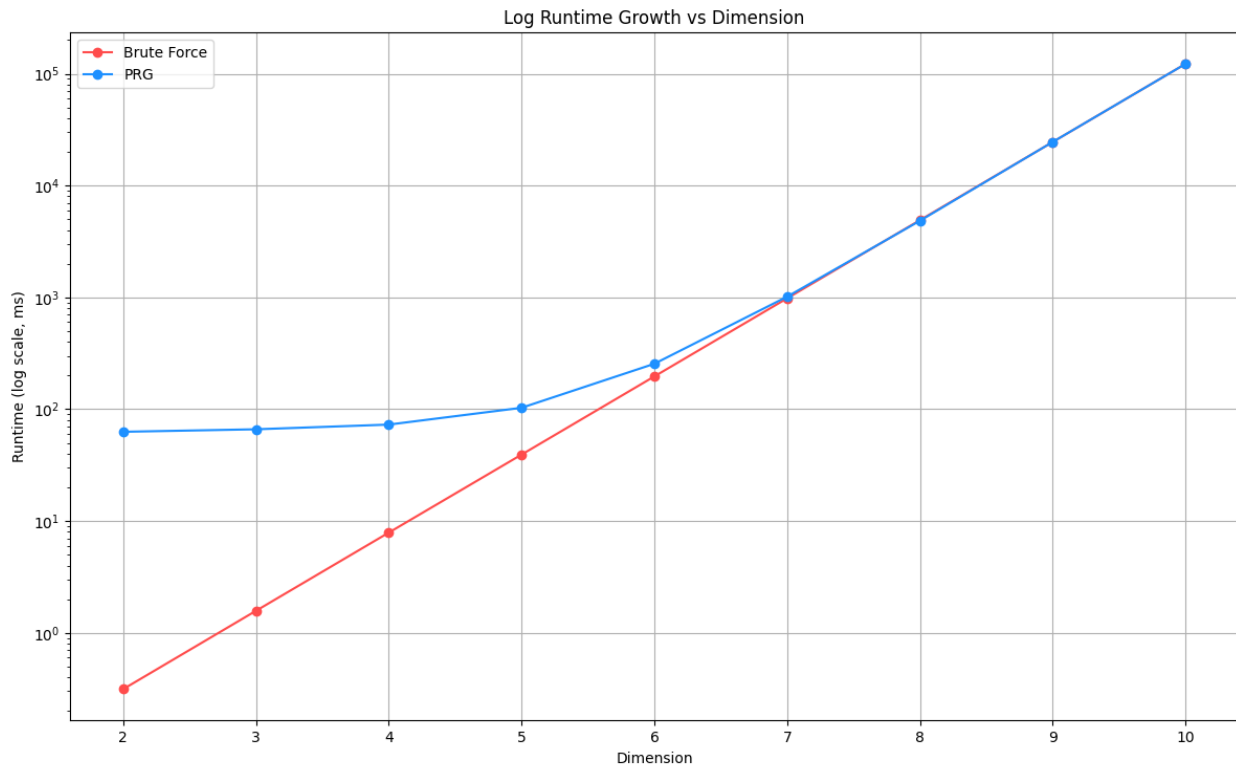


Figure 28: Graph of log runtime growth of PRG and brute-force across dimensions, created by Finalist.

The log-scale runtime graph (Figure 28) displays the growth trend more clearly, showing that both algorithms exhibited exponential increases in runtime as dimension increased.

#### 4.3 Effect of Basis Conditioning

The effect of basis conditioning on the lattice structures was evaluated by comparing the condition scores of each basis before and after reduction. Across all tested lattice dimensions, a substantial decrease in condition number was observed following the implemented basis reduction. The results show that originally ill-conditioned lattices transformed into extremely more orthogonal and structured forms.



*Table 7: Summary of mean basis condition scores and improvement, created by Finalist.*

| Dimension | Mean Score Before | Mean Score After | % Improvement |
|-----------|-------------------|------------------|---------------|
| 2         | 60.728            | 3.495            | 94.24         |
| 3         | 484.163           | 5.749            | 98.81         |
| 4         | 462.323           | 7.315            | 98.42         |
| 5         | 594.567           | 7.857            | 98.68         |
| 6         | 578.502           | 10.109           | 98.25         |
| 7         | 1812.470          | 9.873            | 99.46         |
| 8         | 3371.195          | 14.617           | 99.57         |
| 9         | 2426.054          | 17.409           | 99.28         |
| 10        | 9413.068          | 25.554           | 99.73         |

The magnitude of this improvement increased with dimension with higher-dimensional lattices showing extremely large reductions. With basis reduction implemented, it consistently reduced the condition number by over 94% across all dimensions, with higher-dimensional lattices showing the largest improvements. This indicated that the basis reduction becomes increasingly important as dimensional complexity grows. The reduction in the condition scores suggests that the lattice geometry was able to become less distorted, therefore improving the accessibility of shorter vectors within the search space.

#### 4.4 PRG Outperformance Rate

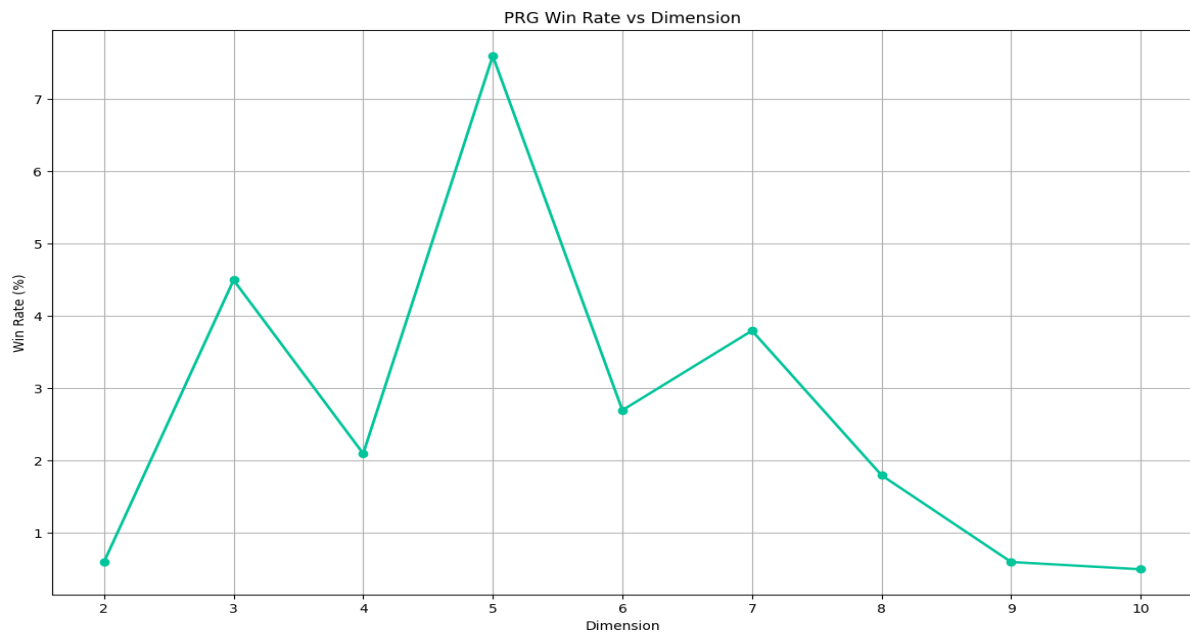


Figure 29: Graph of PRG win/outperformance rate vs dimension, created by Finalist.

The PRG win rate graph (Figure 29) shows the proportion of trials in which PRG produced a shorter vector than brute-force across each tested dimension. The graph indicates that the win rate remains very low across all tested dimensions, with values close to zero in most cases. While small fluctuations were present, there was no consistent increase in win rate with dimensional increase. This confirms that PRG in some rare case produced a strict shorter vector than what was found by brute-force.

#### 4.5 PRG Mean and Relative Error Analysis

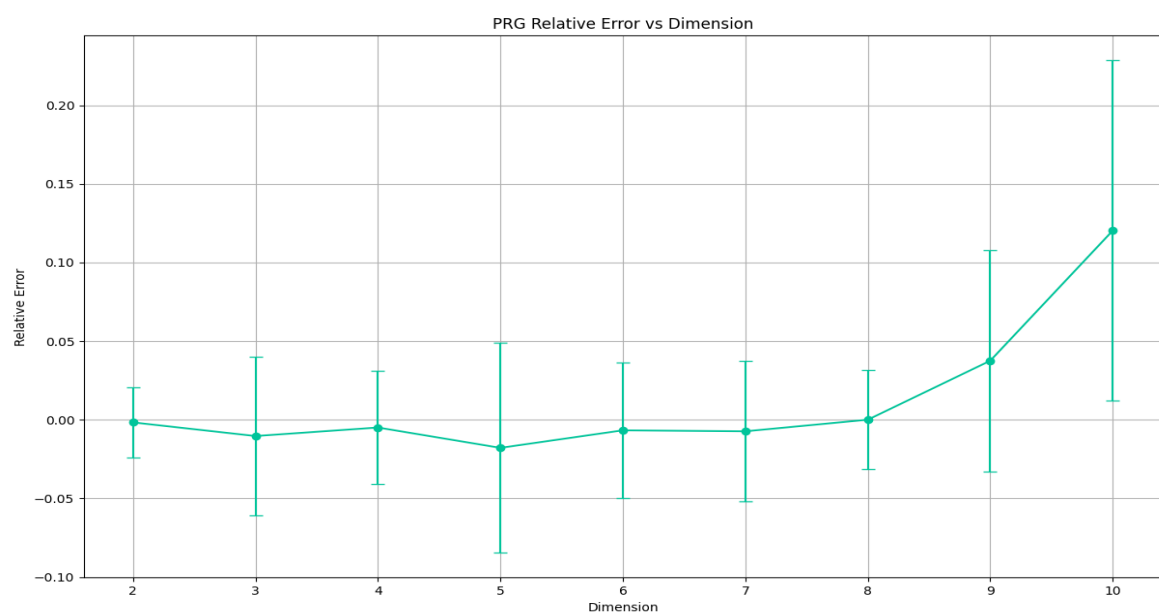


Figure 30: Graph of PRG relative error across dimensions, created by Finalist.

The relative error plot (Figure 30) shows how the difference between PRG and brute-force solutions changes with dimension. The results highlight that relative error remained very low in the lower tested dimensions with values clustered close to zero. Although as dimension increased, both the spread and magnitude of error values increased, signaling greater variability in PRG performance.

Table 8: Summary of mean and absolute errors, created by Finalist.

| Dim | Mean Error | Std Dev | MAE    |
|-----|------------|---------|--------|
| 2   | -0.0017    | 0.0225  | 0.0017 |
| 3   | -0.0103    | 0.0505  | 0.0103 |
| 4   | -0.0049    | 0.0359  | 0.0049 |
| 5   | -0.0179    | 0.0669  | 0.0179 |
| 6   | -0.0067    | 0.0430  | 0.0067 |
| 7   | -0.0073    | 0.0448  | 0.0086 |
| 8   | 0.0000     | 0.0316  | 0.0071 |
| 9   | 0.0376     | 0.0704  | 0.0400 |
| 10  | 0.1203     | 0.1083  | 0.1227 |

The summary table of mean relative and absolute errors supports the observations. This shows an increase in both average error and variability as the lattice dimension increases.

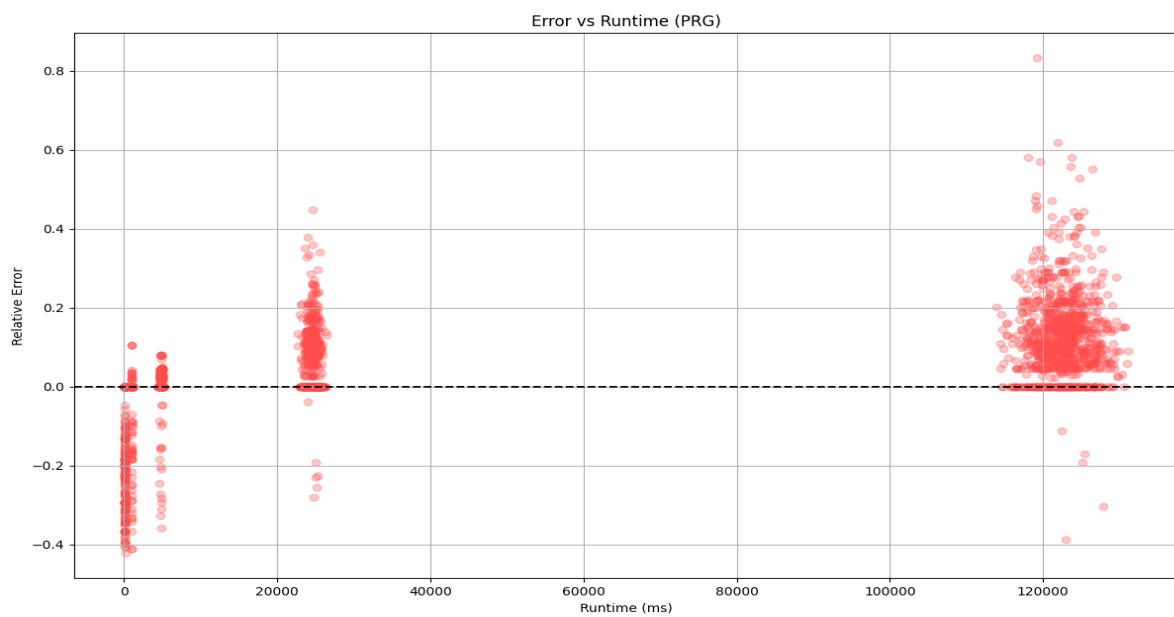


Figure 31: Graph of PRG error vs runtimes, created by Finalist.

The error vs runtime scatter plot (Figure 31) shows the relationship between computation time and solution accuracy for PRG. The distribution of points does not show a strong linear trend, with both low and high error values occurring across a range of runtimes.

#### **4.6 Summary of Observed Patterns**

There were several consistent patterns observed throughout the final experimentation and results:

1. PRG produced vector norms very close to those found by brute-force, with most trials resulting in identical or near-identical solutions.
2. Runtime for both algorithms increased rapidly with dimension, although the growth trend became particularly significant in the higher-dimensional lattices tested.
3. Relative error and variability also increased as dimension increased, which indicated that PRG slightly struggled to keep stable in more complex search spaces.

Overall, these results showed that PRG maintained a high approximation accuracy and quality while demonstrating behaviour consistent with a scalable heuristic approach.

## 5 Analysis and Discussion

The experimental results provided insights into the behaviour and performance of the novel PRG algorithm compared with the brute-force approach for solving the SVP across the range of tested lattice dimensions. Several patterns emerged across the metrics of vector norms, runtime, success rates, and errors of accuracy, each of which can be interpreted in light of existing research.

### 5.1 Vector Norms

Across all tested dimensions, the mean vector norms increased with lattice dimension, displaying the expected expansion of the search space. This aligns with prior findings in lattice-based cryptography, where the length of the shortest vector tends to grow as the rank of the lattice increases (Micciancio & Goldwasser, 2002). The PRG algorithm consistently produced vectors with norms within the range identified by brute-force, occasionally yielding slightly smaller or larger norms. This variability is characteristic of randomised heuristic approaches (Nguyen & Vidick, 2008), where sampling strategies introduce stochastic variation in outcomes.

Table 9: Numerical comparison of norms, created by Finalist.

| Dimension | Mean BF Norm | Mean PRG Norm | Observation                            |
|-----------|--------------|---------------|----------------------------------------|
| 2         | 2.700        | 2.695         | Very close in value.                   |
| 3         | 3.170        | 3.140         | Very close in value.                   |
| 4         | 3.447        | 3.431         | Very close in value.                   |
| 5         | 3.866        | 3.800         | Slight PRG decrease                    |
| 6         | 4.436        | 4.406         | Very close in value.                   |
| 7         | 4.671        | 4.637         | Very close in value.                   |
| 8         | 4.996        | 4.994         | Very close in value, nearly identical. |
| 9         | 5.395        | 5.590         | PRG was slightly higher.               |
| 10        | 5.867        | 6.554         | Larger PRG deviation.                  |

The close accuracy between PRG and brute-force indicated that PRG was capable of reliably approximating the SV in the tested lattice dimensions. These findings are consistent with other heuristic lattice SVP methods, where approximate solutions closely match the optimal values but vary slightly (Nguyen & Vidick, 2008).

### 5.2 Algorithm Runtime

The runtime results showed a clear exponential increase in computational cost as the lattice dimensions increased. Brute-force remained quicker in the lower set of dimensions (2-5), this is due to its simplicity and search space. However, the brute-force runtime grew rapidly which reflects the exponential complexity of exhaustive enumeration. While PRG was observed as slower in the tested dimensions due to overhead from the additional sampling and local search, it demonstrated a more controlled growth rate in comparison.

Table 10: Numerical comparison of runtimes, created by Finalist.

| Dimension | BF Mean (ms) | PRG Mean (ms) | Observation                      |
|-----------|--------------|---------------|----------------------------------|
| 2         | 0.31         | 62.78         | PRG slower with overhead.        |
| 3         | 1.57         | 66.10         | PRG slower with overhead.        |
| 4         | 7.87         | 72.80         | Large runtime gap.               |
| 5         | 39.24        | 102.91        | PRG slower with overhead..       |
| 6         | 196.32       | 254.71        | Still slower, but gap narrowing. |
| 7         | 980.50       | 1014.47       | Similar runtimes.                |
| 8         | 4905.28      | 4843.32       | PRG slightly faster.             |
| 9         | 24534.11     | 24561.46      | Nearly identical.                |
| 10        | 122615.86    | 122534.52     | Nearly identical.                |

The calculated exponential growth base of around 2.66, and an average growth factor of 3.11 per dimension, confirmed that PRG scaled more gradually in comparison to brute-force. The runtime ratio of approximately 28.9 further highlights the overhead present in lower lattice dimensions, but this gap appeared to reduce as the testing dimension increased. These results overall demonstrate the trade-off between exact and heuristic SVP approaches, with the novel PRG exhibiting improved scalability despite higher initial cost.

### 5.3 Basis Conditioning

The application of basis reduction significantly improved the condition of the lattices, therefore reducing the condition number and making the basis vector more orthogonal. This preprocessing step directly impacted the efficiency of the solver by reducing distortion in the lattice geometry. A more well-conditioned basis ensured that shorter vector are more easily obtainable within the search space, improving effectiveness in PRG sampling.

Table 11: Numerical comparison of basis conditioning scores across dimensions, created by Finalist.

| Dimension | Mean Before | Mean After | % Improvement | Observation                                        |
|-----------|-------------|------------|---------------|----------------------------------------------------|
| 2         | 60.728      | 3.495      | 94.24%        | Large reduction.                                   |
| 3         | 484.163     | 5.749      | 98.81%        | Large reduction, mean before significantly higher. |
| 4         | 462.323     | 7.315      | 98.42%        | Large reduction.                                   |
| 5         | 594.567     | 7.857      | 98.68%        | Large reduction, initial score increasing.         |
| 6         | 578.502     | 10.109     | 98.25%        | Large reduction.                                   |
| 7         | 1812.470    | 9.873      | 99.46%        | Very large improvement.                            |
| 8         | 3371.195    | 14.617     | 99.57%        | Very large improvement.                            |
| 9         | 2426.054    | 17.409     | 99.28%        | Very large improvement.                            |
| 10        | 9413.068    | 25.554     | 99.73%        | Highest observed reduction.                        |

This finding aligns with established theory in lattice reduction, where poorly conditioned bases can significantly affect the accuracy and efficiency of SVP solvers (Lenstra, Lenstra, & Lovász, 1982) (Nguyen & Stehlé, 2010). By improving conditioning before applying the PRG algorithm, the likelihood of identifying vectors close to the shortest vector is enhanced, supporting the robustness of the novel approach even in difficult lattice configurations.

A closer examination of trials where PRG produced shorter vectors reveals that these successes often occurred in lattices with higher initial condition numbers. In poorly conditioned bases, the brute-force approach was constrained by its limited coefficient search range, preventing it from exploring vectors with larger coefficients that may yield shorter norms. The PRG solver was able to explore beyond these limitations, on occasion identifying shorter vectors. This pattern is consistent with prior observations in the literature, where other interpretations of randomised heuristics demonstrate an advantage in lattices with ill-conditioned bases, especially when exhaustive search is constrained (Micciancio D. , 2011) (Nguyen & Vidick, 2008). These results showcase a specific strength of the PRG approach; its robustness to variations in lattice conditioning, which may become increasingly relevant in higher, closer to cryptographically-scaled dimensions where exhaustive search becomes infeasible.

#### 5.4 PRG Outperformance Rates

The PRG algorithm rarely, but effectively produced shorter vectors than brute-force, achieving this outcome in only a small fraction of trials per dimension. Although this percentage is small, the ability for PRG to outperform and identify a shorter vector than brute-force (which aims to search the entire lattice within a certain range), is extremely promising and highlights the capability of the novel PRG approach. The development of accurate, scalable, and interpretable heuristics highlights promising directions for PQC and future development.

Table 12: Numerical comparison of PRG success rates, created by Finalist.

| Dimension | Win (%) | Tie (%) | Loss (%) | Observation                                     |
|-----------|---------|---------|----------|-------------------------------------------------|
| 2         | 0.6     | 99.4    | 0.0      | SV almost always a match.                       |
| 3         | 4.5     | 95.5    | 0.0      | Slightly higher PRG win rate.                   |
| 4         | 2.1     | 97.9    | 0.0      | Slightly lower win rate.                        |
| 5         | 7.6     | 92.4    | 0.0      | Slight peak in win rate.                        |
| 6         | 2.7     | 97.3    | 0.0      | SV almost always a match.                       |
| 7         | 3.8     | 94.7    | 1.5      | Mostly ties, losses begin to occur.             |
| 8         | 1.8     | 87.3    | 10.9     | Losses increasing, ties still heavily relevant. |
| 9         | 0.6     | 67.7    | 31.7     | More significant losses.                        |
| 10        | 0.5     | 18.9    | 80.6     | PRG mostly worse (stochastic variability)       |

These results suggest that the PRG algorithm demonstrated potential rather than superiority in low dimensions, in agreement with existing literature (Table 8) (Micciancio D. , 2011).

## 5.5 Errors of Accuracy

Table 13: Mean relative error of PRG compared to brute-force, created by Finalist.

| Dimension | Mean Error | Std Dev | MAE    | Observation                 |
|-----------|------------|---------|--------|-----------------------------|
| 2         | -0.0017    | 0.0225  | 0.0017 | Very low error.             |
| 3         | -0.0103    | 0.0505  | 0.0103 | Slight increase in error.   |
| 4         | -0.0049    | 0.0359  | 0.0049 | Very low error.             |
| 5         | -0.0179    | 0.0669  | 0.0179 | Slight increase in error.   |
| 6         | -0.0067    | 0.0430  | 0.0067 | Very low error.             |
| 7         | -0.0073    | 0.0448  | 0.0086 | Very low error.             |
| 8         | 0.0000     | 0.0316  | 0.0071 | Balanced error.             |
| 9         | 0.0376     | 0.0704  | 0.0400 | Increase in error.          |
| 10        | 0.1203     | 0.1083  | 0.1227 | Highest deviation observed. |

The accuracy of PRG was determined using relative error metrics, to show that the algorithm maintained a high level of precision across all dimensions. The overall accuracy was approximately 97.6%, with a MAE of  $\sim 0.024$ . This indicated that PRG solutions were typically within a very small margin of the optimal solution. With lattice dimensional increase, both the mean error and standard deviation increased which indicated a greater variability in performance. The maximum observed error of  $\sim 0.83$  shows that in exceptionally rare cases, PRG may produce a significantly less optimal solution. However, these types of instances were infrequent and the overall distribution remained centred tightly around zero. Overall, these findings confirmed that the novel PRG approach maintained strong accuracy while being exposed to a range of different lattice structures across a wide range of dimensions, with its variability increasing being a typical expectation in stochastic algorithms.



## 5.6 Interpretation and Comparison

Overall, the observed patterns across the final results confirm that the PRG algorithm behaved as a heuristic should as it produced competitive solutions within the expected norm range, incurs additional computation overhead in low-dimensional cases, and benefits from improved basis conditioning. Differences in observed PRG success rates compared to other heuristic studies may arise from the specific lattice generation method, the lattice dimensions used, and the search parameters applied to brute-force enumeration.

In summary, the novel PRG algorithmic approach to the SVP demonstrates promise as a heuristic, showing strengths in consistency, robustness to conditioning, and occasional identification of competitive short vectors. These outcomes highlight the potential of this unique approach to complement traditional SVP solvers, especially in higher-dimensional lattices where exhaustive search is infeasible.

## 6 Threats to Validity (Limitations and Errors)

This section lists and explains the things that constrained measurements and could have biased results in this project. For each item, the cause, the mathematical effect, and effect on results are provided.

### 1. Experimental Scale and Generalisability:

Tests were performed on small ranks ( $n \leq 10$ ). Lattice cryptography and provable hardness rely on behaviour in high dimensions, which can extend to hundreds or thousands. Complexity and geometry of lattices change qualitatively with dimension; enumeration and pruning strategies that work in small dimensions may not scale or may behave differently in large dimensions. Claims about cryptographic security or runtime scaling must therefore be restricted to the tested range. This is a fundamental threat to validity.

Observed runtime improvements and solution quality for PRG may not persist at substantially higher dimensions. Metrics such as median runtime, success rates and variance are only valid for the tested small dimensions.

### 2. Bounded Brute-force Baseline (Comparison Fairness):

The brute-force solver was bounded to coefficient ranges  $[-r, r]^n$ . PRG sampling used a different coefficient range. If the true shortest vector has coefficients outside the brute-force bound, the solver will not find it. PRG may appear to exceed even though a larger range brute-force could find the same vector or a better vector. Several observed outperformances by PRG can be explained by brute-force being artificially limited. It does not fully prove PRG beats unbounded brute-force (which is infeasible). It therefore shows PRG's ability to explore different coefficient regions more efficiently given the parameter choices.

### 3. Numerical and Rounding Errors (Floating-point Arithmetic):

Many operations used floating-point arithmetic (dot products, eigenvalue computation for condition score, Gram-Schmidt steps, PCA projections). Rounding, limited precision, and casting to integers can introduce numerical errors. Numerical linear algebra literature documents many pitfalls. Eigenvalue computations can produce tiny negative eigenvalues due to rounding; taking the ratio without guarding against near-zero or negative values given unstable condition scores. In the basis reduction, rounding and converting to integers can produce non-unimodular transformations (numerical drift). PCA projection changes relative distances and may distort perceived vector lengths. PCA was only used for visualisation, not a correctness test.

Condition scores may be noisy for nearly dependent bases. PCA visualisations could give misleading impressions about vector lengths and relative positions. In extreme cases small numerical errors could cause a reported zero-norm vector (anomalous results noted in earlier runs).

#### 4. Randomness, Sampling Variance, and Statistical Uncertainty:

PRG is randomised. Results like norm and coefficients varied with RNG seed and number of trials  $T$ . A single-run experiment is inadequate to characterise performance. PRG performance metrics are random variables. Point estimates (single-run norms or runtimes) can be misleading and distributions can be skewed.

Variance across repeated PRG runs can be large and single-run success may be due to chance. Reported efficiency is valid only in expectation or with confidence intervals.

#### 5. Timing, Benchmarking, and Hardware Dependencies:

Wall-clock timings used to compare runtimes depend on the machine, background processes, BLAS/MKL acceleration and number of threads. Python/NumPy BLAS libraries can change observed runtimes dramatically.

For example, two runs of identical code can differ in runtime due to OS scheduling or multi-threaded BLAS. This is especially relevant for dense linear algebra used in basis operations and PCA.

#### 6. Visualisation and Interpretive Errors (PCA):

PCA was used to create 3D visualisations for high-dimensional lattices. PCA projects points onto principal components that preserve variance, not exact pairwise Euclidean distances. Two vectors that appear close in PCA space may be distant in original space, and vice versa. A reader could incorrectly infer performance or vector length from the plot alone.

#### 7. Implementation Correctness and Logging:

Bugs and unintended behaviours can affect results, such as a missing guard for *best\_coeffs* is *None* in PRG local search (which was fixed), incorrect treatment of all-zero coefficient vector, or errors off by one in iteration bounds. Crashes or incorrect outputs such as a zero norm or other problems around coefficient reporting were observed in early runs.

#### 8. Choice of Random Basis Distribution and Problem Representativeness:

Lattice bases were sampled entry-wise from a uniform integer range (e.g.,  $[-5, 5]$ ). This choice biased the class of lattices being tested. Some lattice families (e.g., q-ary lattices or struted cryptographic lattices) have very different geometry. Results on uniformly random small-entry bases do not necessarily generalise.

## 7 Future Work and Areas of Improvement

While this project successfully implemented and compared the novel PRG approach to the SVP, there remain several areas where the work could be improved and extended in the future.

A key advancement achieved in this project was the complete implementation of a basis reduction algorithm. Further improvements could involve optimising this implementation or integrating established high-performance libraries (like `fpvll`), which would provide stronger numerical stability and closer alignment with state-of-the-art cryptographic tooling (Lenstra, Lenstra, & Lovász, 1982) (Nguyen & Vallée, 2010). This would also allow for more efficient handling of higher dimensional lattices for further testing.

Beyond this, exploring the architecture of stronger reduction techniques such as BKZ or deep insertion variants presents significant opportunity. These methods produce higher-quality reduced bases which are critical in modern PQC, although they often come with increased computational cost (Chen & Nguyen, 2011). Incorporating these techniques would enable a more realistic evaluation of solver performance under cryptographic conditions.

Scaling the PRG algorithm testing to higher dimensions is another important and promising direction. While the project extended testing up to  $n = 10$  with large trial counts, practical cryptographic lattices usually operate in dimensions ranging from hundreds to thousands (Micciancio & Goldwasser, 2002). Adapting the current algorithms to operate efficiently at these scales and potentially through parallelisation, GPU acceleration, or more efficient data structures would immensely enhance the real-world applicability of the work.

Further refinement of the PRG algorithm itself is a major and promising area. With the current implementation involving a novel combination of randomised sampling, adaptive pruning, local search refinement, a promising direction could be exploring other optimisation strategies. This includes metaheuristic techniques such as simulated annealing, evolutionary algorithms, or adaptive parameter tuning which may improve exploration of the search space and reduce its sensitivity to initial sampling.

Additionally, the theoretical modelling of the algorithm performance could be extended significantly. This project identified a gap between classical asymptomatic complexity models and the observed behaviour in tested dimensions. Developing hybrid models that can combine theoretical bounds with empirical calibration would give a more accurate prediction framework for modelling algorithmic scalability.

Finally, improvements in statistical analysis would strengthen the robustness of the results. Even though large numbers of trials were conducted, further work could include a deeper statistical testing, confidence intervals, and hardware-level benchmarking. This would ensure that conclusions regarding runtime and accuracy are both reproducible and independent of implementation-specific factors.

## 8 Conclusions and Applications

This project successfully explored the development and evaluation of the novel PRG algorithm for solving and approximating the SVP. The main objective was to design a novel heuristic approach capable of producing competitive short vectors while maintaining robustness to basis conditioning. Through the integration of lattice preprocessing, random sampling, pruning and local search, the project established a structured and efficient approach to exploring lattice solution spaces.

### Vector Norm Performance:

The experimental results showed that PRG consistently produced vector norms within the same range as those identified by brute-force enumeration. In a small number of cases and particularly in those of poorly-conditioned lattices, PRG was able to identify slightly shorter and more optimal results than the brute-force approach. This highlights that the probabilistic nature of PRG allows it to explore regions of lattices that may not be fully covered by enumeration. Overall this demonstrated the capability of PRG in generating competitive and reliable approximations to the true SV.

### Algorithmic Runtime and Scalability:

In the tested dimensional settings, PRG exhibited a slightly higher runtime compared to brute-force search. This is expected due to the additional computational overhead. However, runtime analysis also revealed that PRG scales more effectively as dimension increases. While enumeration grows exponentially, PRG demonstrates a slower sub-exponential effective growth rate due to its ability to prune large portions of the lattice search space. This also suggests that PRG becomes increasingly advantageous as the dimensionality grows, which aligns with theoretical expectations for heuristic SVP solvers.

### Effect of Basis Conditioning:

The implementation of preprocessing immensely improved the conditioning of the lattice bases across all tested dimensions. Condition number reductions of over 94% were observed, which indicated a substantial improvement in lattice orthogonality. It had a direct impact on solver performance by reducing the effective search radius, improving numerical stability, and increasing the likelihood of sampling shorter vectors. These findings confirm that preprocessing lattices is a critical component in practical SVP solving, reinforcing its role in both theoretical and applied algorithms.

### PRG Success and Outperformance Behaviour

The PRG algorithm rarely but occasionally outperformed brute-force in the tested dimensions, which is expected given that brute-force explores the majority of the search space. However, when PRG did achieve better results, this occurred primarily in poor conditioned lattices or cases where brute-force was limited by its bounded nature. This behaviour highlights an important property of heuristic

algorithms where they do not guarantee optimality, but can occasionally discover better solutions through non-deterministic exploration. PRG demonstrated better potential in the tested dimensions, rather than superiority, with its strengths becoming even more relevant as the search space expands.

### **Accuracy and Reliability:**

Error analysis displayed that PRG maintained a high level of accuracy overall with low can and absolute error values. Variability still existed due to stochastic nature, but the results remained consistent across a large number of testing trials. This indicates that PRG was stable across repeated runs, robust to randomness, and capable of producing reliable SV approximations. This consistency is essential for practical applications where deterministic guarantees are not always feasible.

### **Applications in SVP and PQC:**

The broader significance of this work lies in its relevance to lattice-based cryptography and PQC. The harness of the SVP underpins many modern cryptographic constructions including lattice-based encryption schemes, digital signatures, and fully homomorphic encryption. In these areas, efficient algorithms for exploring lattice structures are essential for cryptanalysis to test security strength, and algorithmic design for optimising implementations. The novel PRG approach contributed to this area by demonstrating how it can efficiently explore a range of dimensional solution spaces and provide a scalable alternative in areas where exhaustive methods are impractical. While the algorithm does not threaten cryptographic security in its current stage, it reflects the type of optimisation strategies that must be considered when evaluating the long-term resilience of lattice-based systems.

Overall, this project conclusively demonstrated that a novel pruned, randomised and locally refined heuristic SVP solver can give a powerful framework for solving lattice problems. Although brute-force methods are optimal in very low dimensions, PRG offers a clear advantage in scalability and adaptability. Importantly, the work and research also highlights the gap between theoretical complexity models and practical performance, showing that real-world implementations can behave significantly differently from asymptomatic predictions. The findings largely support the initial hypothesis, showing that the PRG approach can generate near-optimal vectors with competitive accuracy, while exhibiting characteristics that indicate greater efficiency as the lattice dimensions increase.

## 9 Acknowledgements

I would like to acknowledge the help I received from:

- My parents for always encouraging me in my exploration of STEM and Computer Science. They have always supported me and make sure I have everything I need.
- My teacher and project mentor, Zita Murphy, for her feedback, mentorship, and direction throughout the course of this project.
- Sheila Porter, George Porter and the SciFest team for their encouragement, and for creating opportunities to present and develop this project further. The support of SciFest CLG in fostering young scientific research in Ireland is deeply appreciated.
- My school for their encouragement and constant support throughout my research, and their engagement throughout participation in scientific competitions. In particular, I would like to give appreciation to my Deputy Principal, Emma Gleeson, for her incredible support and enthusiasm in all of my projects.
- The wide range of content creators online whose educational resources, tutorials, and discussions provided inspiration and technical insight during the development of the project.
- Dr. Hitesh Tewari from Trinity College Dublin and Dr. Mel Ó Cinnéide from UCD, for providing some suggestions from reviewing my project report. Their guidance, expertise, and constructive feedback, which greatly strengthened the project's depth and clarity.
- Róisín Faherty, Karen Nolan and Keith Quille from TU Dublin for allocating time for me to present this project, and their incredible feedback and advice given in return.
- The Society for Science for all of their hard work and for providing helpful and constructive guidance for completing an ISEF project. Their support has been truly remarkable.

The completion of this project would not have been possible without the collective support of these individuals and organisations.

## 10 References and Image Credits

- Ajtai, M. (1996). Generating hard instances of lattice problems. In Proc. of the 28th Annual ACM Symposium on Theory of Computing. STOC 1996.
- Ajtai, M., & Dwork, C. (1997). Proceedings of the twenty-ninth annual ACM symposium on Theory of computing. *A public-key cryptosystem with worst-case/average-case equivalence.* , 284–293. Retrieved from <https://doi.org/10.1145/258533.258604>
- Ajtai, M., Kumar, R., & Sivakumar, D. (2001). STOC. *A sieve algorithm for the shortest lattice vector problem.*
- Albrecht, M. R., Gheorghiu, V., Postlethwaite, E. W., & Schanck, J. M. (2020, December). In International Conference on the Theory and Application of Cryptology and Information Security. *Estimating quantum speedups for lattice sieves.*, 583-613. Springer International Publishing.
- Aono, Y., Nguyen, P. Q., & Seito, T. (2018). Designs, Codes and Cryptography. *Faster Lattice Enumeration Using Extreme Pruning.*
- Arora, S., Babai, L., Stern, J., & Sweedyk, Z. (1997). Journal of Computer and System Sciences. *The hardness of approximate optima in lattices, codes, and systems of linear equations.* , 54(2), 317–331. Retrieved from <https://doi.org/10.1006/jcss.1997.1472>
- Babai, L. (1986). Combinatorica. *On Lovász' lattice reduction and the nearest lattice point problem.*
- Bernstein, D., Buchmann, J., & Dahmen, E. (2009). Post-Quantum Cryptography. . Berlin: Springer.
- Campbell, P., Groves, M., & Shepherd, D. (2020). In Proceedings of Workshop on Post-Quantum Cryptography . PQCrypto.
- Chen, Y., & Nguyen, P. (2011). In Advances in Cryptology – ASIACRYPT 2011. *BKZ 2.0: Better lattice security estimates.* , pp.1–20. Springer.
- Chen, Y., & Nguyen, P. Q. (2011). Advances in Cryptology – ASIACRYPT 2011. *BKZ 2.0: Better lattice security estimates.* , vol 7073. Heidelberg, Berlin: Springer. Retrieved from [https://doi.org/10.1007/978-3-642-25385-0\\_24](https://doi.org/10.1007/978-3-642-25385-0_24)
- Fowler, M. a. (2001). "The agile manifesto." . *Software development* 9, 8, 28-35.
- Gama, N. &. (2010). Finding short lattice vectors within Mordell's inequality. *STOC.*
- Gama, N., & Nguyen, P. (2008). In Advances in Cryptology – EUROCRYPT 2008. *Predicting lattice reduction.*, vol 4965, pp.31–51. Heidelberg, Berlin: Springer. Retrieved from [https://doi.org/10.1007/978-3-540-78967-3\\_12](https://doi.org/10.1007/978-3-540-78967-3_12)
- Gama, N., Nguyen, P., & Regev, O. (2010, June). EUROCRYPT 2010. *Lattice Enumeration using Extreme Pruning.* Springer.
- Goldreich, O., Goldwasser, S., & Halevi, S. (1997). Advances in Cryptology – CRYPTO'97. *Public-key cryptosystems from lattice reduction problems.*, vol 1294. Heidelberg, Berlin: Springer. Retrieved from <https://doi.org/10.1007/BFb0052231>
- Hanrot, G., Pujol, X., & Stehlé, D. (2011). CRYPTO. *Analyzing blockwise lattice algorithms using dynamical systems.* .



- Hanrot, G., Pujol, X., & Stehlé, D. (2011). Third International Workshop, IWCC 2011. *Algorithms for the shortest and closest lattice vector problems*. In *Coding and Cryptology*, pp.159–190. Springer.
- Jolliffe, I., & Cadima, J. (2016). Principal component analysis: a review and recent developments. *Phil. Trans. R. Soc. A*.
- Kannan, R. (1987 ). Mathematics of Operations Research. *Minkowski's convex body theorem and integer programming*. , 12(3), pp.415–440.
- Kannan, R. (1987). Mathematics of Operations Research. *Minkowski's convex body theorem and integer programming*.
- Lenstra, A. K., Lenstra, H. W., & Lovász, L. (1982). *Mathematische Annalen*. *Factoring polynomials with rational coefficients*., 261(4), 515–534. Retrieved from <https://doi.org/10.1007/BF01457454>
- Lindner, R., & Peikert, C. (2011). Better key sizes (and attacks) for LWE-based encryption. . *CT-RSA*.
- Micciancio, D. (2011). Heuristics for Lattice Problems: Randomized Algorithms and Sieve Methods.
- Micciancio, D., & Goldwasser, S. (2002 ). Complexity of Lattice Problems: A Cryptographic Perspective. Springer.
- Micciancio, D., & Regev, O. (2007). *SIAM J. Computing*. *Worst-case to average-case reductions based on Gaussian measures*. .
- Micciancio, D., & Regev, O. (2009). Lattice-based cryptography. In *Theory and Applications of Cryptographic Techniques. Eurocrypt 2009*, pp. 147–191 . Heidelberg , Berlin: Springer. Retrieved from [https://doi.org/10.1007/978-3-540-88702-7\\_5](https://doi.org/10.1007/978-3-540-88702-7_5)
- Micciancio, D., & Voulgaris, P. (2010). . In *Proceedings of the 42nd ACM Symposium on Theory of Computing (STOC '10)*. *A deterministic single exponential time algorithm for most lattice problems based on Voronoi cell computations*, pp.351–358.
- Nguyen, P. Q., & Stehlé, D. (2010). LLL on the Average.
- Nguyen, P. Q., & Stern, J. (2001). Cryptography and lattices. *CALC 2001. The two faces of lattices in cryptology*. In *J. H. Silverman (Ed.)*, , vol 2146. Heidelberg, Berlin: Springer. Retrieved from [https://doi.org/10.1007/3-540-44670-2\\_1](https://doi.org/10.1007/3-540-44670-2_1)
- Nguyen, P. Q., & Vallée, B. (2010). *The LLL Algorithm: Survey and Applications*. . Springer.
- Nguyen, P. Q., & Vidick, T. (2008). *J. Math. Cryptology*. *Sieve algorithms for the shortest vector problem are practical*.
- Nguyen, P. Q., & Vidick, T. (2008). *Sieve Algorithms for the Shortest Vector Problem*.
- NIST. (2016). *Post-Quantum Cryptography: NIST's Plan for the Future*. Retrieved August 28, 2025, from <https://csrc.nist.gov>
- NIST. (2024). *Post-Quantum Cryptography Standardization*. U.S. National Institute of Standards and Technology.
- Peikert, C. (2016). Lattice cryptography and its applications. In *Theory of Cryptography Conference . TCC 2016*. Springer.

- Regev, O. (2005). Journal of the ACM. *On lattices, learning with errors, random linear codes, and cryptography.*, 84–93. Retrieved from <https://doi.org/10.1145/1060590.1060603>
- Schnorr, C. P., & Euchner, M. (1994). Mathematical Programming. *Lattice basis reduction: Improved practical algorithms and solving subset sum problems.*
- Schnorr, C., & Euchner, M. (1994 ). Mathematical Programming. *Lattice basis reduction: Improved practical algorithms and solving subset sum problems.*, 66(1–3), pp.181–199.
- Shlens, J. (2014, April 7). Google Research. *A Tutorial on Principal Component Analysis.*
- Shor, P. (1997 ). SIAM Journal on Computing. *Polynomial-Time Algorithms for Prime Factorization and Discrete Logarithms on a Quantum Computer.*, 26(5), pp.1484–1509.
- Shor, P. W. (1994). Algorithms for quantum computation. *Discrete logarithms and factoring*, 124–134. Retrieved from <https://doi.org/10.1109/SFCS.1994.365700>

- Fig 1: Diagram created by Finalist using Matplotlib
- Fig 2: Diagram created by Finalist using PowerPoint
- Fig 3: Flowchart created by Finalist using PowerPoint
- Fig 4: Screenshot of code output taken by Finalist.
- Fig 5: Flowchart created by Finalist using PowerPoint.
- Fig 6: Screenshot of PCA projected lattice taken by Finalist.
- Fig 7: Screenshot of code output taken by Finalist.
- Fig 8: Flowchart created by Finalist using PowerPoint.
- Fig 9: Diagram created by Finalist using Matplotlib.
- Fig 10: Plot created by Finalist using Plotly.
- Fig 11: Screenshot of code output taken by Finalist.
- Fig 12: Flowchart created by Finalist using PowerPoint.
- Fig 13: Screenshot of lattice interface taken by Finalist.
- Fig 14: Screenshot of code output taken by Finalist.
- Fig 15: Graph created by Finalist using Matplotlib.
- Fig 16: Graph created by Finalist using Matplotlib.
- Fig 17: Graph created by Finalist using Matplotlib.
- Fig 18: Graph created by Finalist using Matplotlib.
- Fig 19: Graph created by Finalist using Matplotlib.
- Fig 20: Graph created by Finalist using Matplotlib.
- Fig 21: Graph created by Finalist using Matplotlib.
- Fig 22: Graph created by Finalist using Matplotlib.
- Fig 23: Graph created by Finalist using Matplotlib.
- Fig 24: Screenshot of code output taken by Finalist.
- Fig 25: Graph created by Finalist using Matplotlib.
- Fig 26: Graph created by Finalist using Matplotlib.
- Fig 27: Graph created by Finalist using Matplotlib.
- Fig 28: Graph created by Finalist using Matplotlib.
- Fig 29: Graph created by Finalist using Matplotlib.
- Fig 30: Graph created by Finalist using Matplotlib.
- Fig 31: Graph created by Finalist using Matplotlib.

## 11 Appendices

### Appendix 1: Full Final PRG Algorithm Code

Below is the full final code of iteration 4. The full code from previous iterations can be viewed published on my GitHub repository.

The GitHub repository can be accessed through searching: addicarey/LatticeSVP

```
#imports AC
import numpy as np
import time
from sklearn.decomposition import PCA
import plotly.graph_objs as go
from tabulate import tabulate

#basis reduction AC
def basis_reduction(B, delta=0.75):
    B = B.copy().astype(float)
    n = B.shape[1]

    # gram schmidt orthogonalisation AC
    def gram_schmidt(B):
        G = np.zeros_like(B)
        mu = np.zeros((n, n))
        for i in range(n):
            G[:, i] = B[:, i]
            for j in range(i):
                mu[i, j] = np.dot(B[:, i], G[:, j]) / np.dot(G[:, j], G[:, j])
                G[:, i] -= mu[i, j] * G[:, j]
            return G, mu

    k = 1
    while k < n:
        G, mu = gram_schmidt(B)
        # size reduction AC
        for j in range(k - 1, -1, -1):
            if abs(mu[k, j]) > 0.5:
                B[:, k] -= round(mu[k, j]) * B[:, j]
                G, mu = gram_schmidt(B)
        # L condition AC
        if np.dot(G[:, k], G[:, k]) >= (delta - mu[k, k-1]**2) * np.dot(G[:, k-1], G[:, k-1]):
            k += 1
        else:
            B[:, [k, k-1]] = B[:, [k-1, k]] # swap the columns AC
            k = max(k - 1, 1)

    return B.astype(int)
```

```

#condition score AC
def basis_condition_score(B):
    B_T_B = B.T @ B
    eigvals = np.linalg.eigvals(B_T_B)
    return np.max(eigvals).real / np.min(eigvals).real

#brute-force solver AC
def bf_svp_solver(B, max_range=1):
    n = B.shape[1]
    shortest_vec = None
    min_norm = float('inf')
    best_coeffs = None
    for coeffs in np.ndindex*((2 * max_range + 1,) * n):
        coeffs = np.array(coeffs) - max_range
        if np.all(coeffs == 0):
            continue
        vec = B @ coeffs
        norm = np.linalg.norm(vec)
        if norm < min_norm:
            min_norm = norm
            shortest_vec = vec
            best_coeffs = coeffs
    return shortest_vec, best_coeffs, min_norm

#PRG solver AC
def PRG_svp_solver(B, max_trials=10000, initial_radius=10.0):
    n = B.shape[1]
    best_vec = None
    best_norm = float('inf')
    best_coeffs = None
    radius = initial_radius

    for trial in range(max_trials):
        # random sampling AC
        coeffs = np.random.normal(0, 1.5, size=n)
        coeffs = np.round(coeffs).astype(int)
        if np.all(coeffs == 0):
            continue

        vec = B @ coeffs
        norm = np.linalg.norm(vec)

        if norm < radius:
            if norm < best_norm:
                best_norm = norm
                best_vec = vec
                best_coeffs = coeffs
                radius = norm * 1.1 # tight decay AC

```

```

    if best_coeffs is None:
        return None, None, float('inf')

    # extended local search to check +-1 and +-2 neighbours AC
    for delta in np.ndindex(*(5,) * n)):
        offset = np.array(delta) - 2 # -2 to +2 AC
        neighbour = best_coeffs + offset
        if np.all(neighbour == 0):
            continue
        vec = B @ neighbour
        norm = np.linalg.norm(vec)
        if norm < best_norm:
            best_norm = norm
            best_vec = vec
            best_coeffs = neighbour

    return best_vec, best_coeffs, best_norm

#lattice Setup AC
n = 4 #dimensions
B = np.random.randint(-5, 6, size=(n, n))
print("Original Basis Matrix B:\n", B)
original_score = basis_condition_score(B)
print(f"Condition Score (original): {original_score:.2f}")

#apply reduction prep AC
B_reduced = basis_reduction(B)
print("\nReduced Basis Matrix B:\n", B_reduced)
reduced_score = basis_condition_score(B_reduced)
print(f"Condition Score (reduced): {reduced_score:.2f}")

#brute-force solver AC
start = time.time()
shortest_naivebf, coeffs_naivebf, norm_naivebf = bf_svp_solver(B_reduced,
max_range=1)
time_naivebf = time.time() - start

#PRG AC
start = time.time()
shortest_prg, coeffs_prg, norm_prg = PRG_svp_solver(B_reduced)
time_prg = time.time() - start

approx_ratio = norm_prg / norm_naivebf if norm_naivebf > 0 else float('inf')

#results table AC
table = [
    ["Brute-force", str(coeffs_naivebf),
f"{norm_naivebf:.3f}", f"{time_naivebf:.4f} s"],

```

```

    ["PRG", str(coeffs_prg),      f"{norm_prg:.3f}",      f"{time_prg:.4f}
s"],
]

print(tabulate(table, headers=["Method", "Coefficients", "Norm", "Runtime"]))
print(f"\nApproximation Ratio (PRG / BF): {approx_ratio:.4f}")

#PCA visualisation AC
#generate lattice points AC
sample_vectors = []
max_range = 3 # slightly larger range for clarity AC

for coeffs in np.ndindex(*((2*max_range+1,) * n)):
    coeffs = np.array(coeffs) - max_range
    sample_vectors.append(B_reduced @ coeffs)

sample_vectors = np.array(sample_vectors)

#reduce to 3D with PCA AC
pca_3d = PCA(n_components=3)
data_3d = pca_3d.fit_transform(sample_vectors)

naive_proj = pca_3d.transform(shortest_naivebf.reshape(1, -1))
prg_proj = pca_3d.transform(shortest_prg.reshape(1, -1))

#plot AC
fig = go.Figure()

#lattice points AC
fig.add_trace(go.Scatter3d(
    x=data_3d[:, 0], y=data_3d[:, 1], z=data_3d[:, 2],
    mode='markers', marker=dict(size=2, color='blue'),
    name='Lattice Points'
))

#brute-force shortest vector AC
fig.add_trace(go.Scatter3d(
    x=[0, naive_proj[0, 0]],
    y=[0, naive_proj[0, 1]],
    z=[0, naive_proj[0, 2]],
    mode='lines+markers',
    marker=dict(size=5, color='red'),
    line=dict(width=4, color='red'),
    name='Brute-Force SV'
))

#PRG shortest vector AC
fig.add_trace(go.Scatter3d(
    x=[0, prg_proj[0, 0]],

```

```
y=[0, prg_proj[0, 1]],
z=[0, prg_proj[0, 2]],
mode='lines+markers',
marker=dict(size=5, color='green'),
line=dict(width=4, color='green'),
name='PRG + Local Search SV'
))

fig.update_layout(title='PRG and brute-force SVP solutions',
                  scene=dict(xaxis_title='PCA 1',
                             yaxis_title='PCA 2',
                             zaxis_title='PCA 3'),
                  width=1000, height=1000)

fig.show()
```

## Appendix 2: Full Codes for Mathematical Interpretations

```
import numpy as np
np.set_printoptions(precision=4, suppress=True)

#data AC
X = np.array([
    [2, 3, 1, 4, 0],
    [3, 2, 0, 5, 1],
    [4, 4, 2, 6, 2],
    [1, 3, 1, 3, 0]
], dtype=float)

#centring data AC
mu = X.mean(axis=0)
Xc = X - mu

#covariance AC
C = (Xc.T @ Xc) / (X.shape[0] - 1)

#eigen-decomposition AC
eigvals, eigvecs = np.linalg.eigh(C)

#sort descending AC
idx = np.argsort(eigvals)[::-1]
eigvals = eigvals[idx]
eigvecs = eigvecs[:, idx]

#top 3 components AC
V3 = eigvecs[:, :3]

#projection AC
```

```

Y = Xc @ V3

#outputs AC
print("Mean mu:", mu)
print("Covariance C:\n", C)
print("Eigenvalues (desc):", eigvals)
print("Top 3 eigenvectors (columns):\n", V3)
print("Projected Y (4x3):\n", Y)

```

```

#imports AC
import numpy as np
import matplotlib.pyplot as plt

#params AC
alpha = 1.05
omega = 0.5
n_max = 4000

#log funcs AC
def logBF(n): #brute-force
    return n * np.log(alpha)

def logPRG(n): #PRG
    R = alpha * np.sqrt(n)          # GH scaling
    return 2*(np.log(2*R)**2) / abs(np.log(omega))

#domain AC
n_vals = np.arange(2, n_max+1)
bf_vals = logBF(n_vals)
prg_vals = logPRG(n_vals)

#find crossover BF > PRG AC
diff = bf_vals - prg_vals
idx = np.where(diff > 0)[0]
if len(idx) > 0:
    n0 = n_vals[idx[0]]
    print("crossover n0 =", n0)
else:
    n0 = None
    print("no crossover in this dimension range.")

#plot scaling AC
plt.figure(figsize=(11,6))
plt.plot(n_vals, bf_vals, 'r', label="Brute Force: n ln(α)") #BF
plt.plot(n_vals, prg_vals, 'b', label="PRG Bound") #PRG

if n0 is not None:
    plt.scatter([n0], [prg_vals[idx[0]]], color='black')

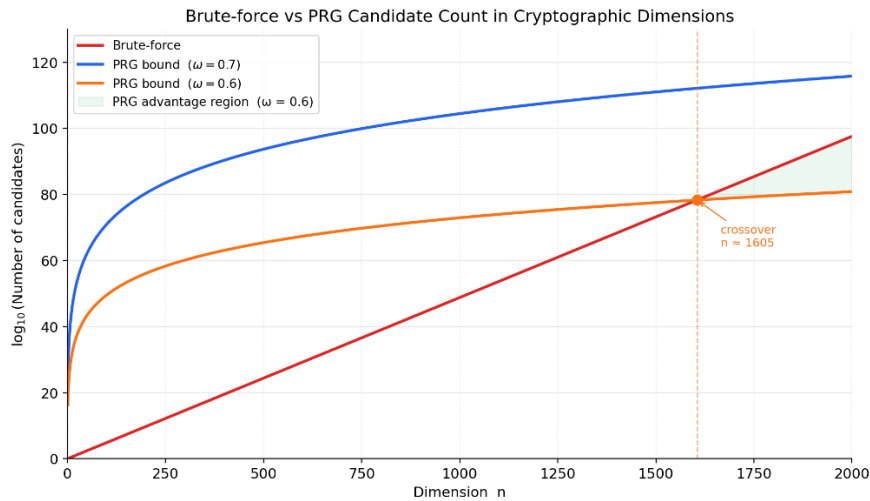
```



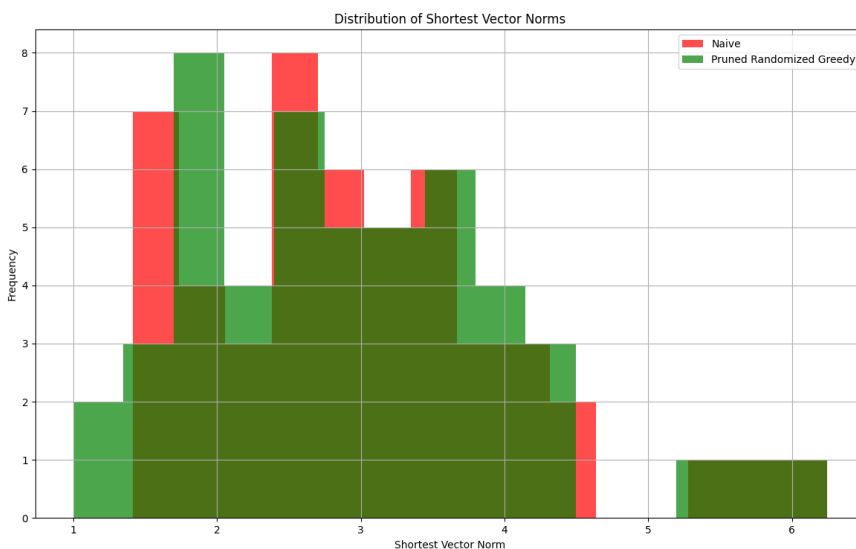
```
plt.annotate(f"Crossover  $n \approx \{n_0\}$ ",
            xy=(n0, prg_vals[idx[0]]),
            xytext=(n0 + 200, prg_vals[idx[0]] + 10),
            arrowprops=dict(arrowstyle="->"))

plt.xlabel("Dimension n")
plt.ylabel("log(Number of Candidates)")
plt.title(f"Brute-Force vs PRG Complexity ( $\omega = \{\omega\}$ )")
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()
```

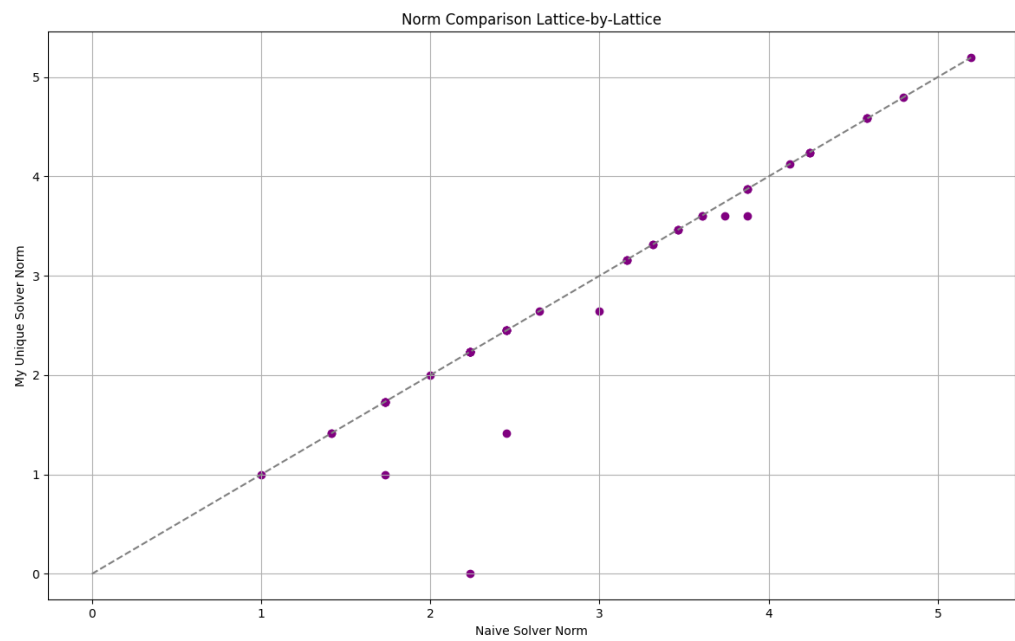
### Appendix 3: Additional Graphs and Results.



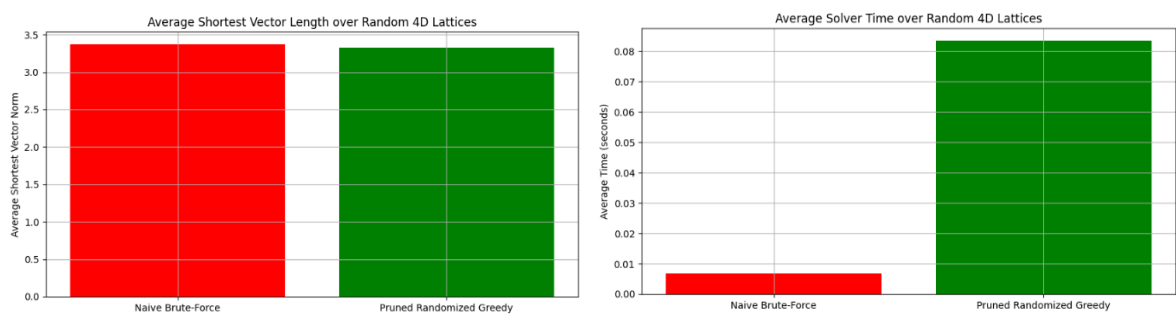
Graph of PRG predicted crossover points, created by Finalist using Matplotlib.



Graph of SV norm distribution (iteration 2), created by Finalist using Matplotlib.



Graph of SV norm comparison (iteration 2), created by Finalist using Matplotlib.



Graph of average SV length comparison and Average runtime comparison (iteration 2), created by Finalist using Matplotlib.